

基于生产过程方法的企业运营体系构建

创建日期：2026-07-15

修订日期：2026-07-18

说明：以生产过程方法构建企业运营体系——从底层逻辑到落地手册。EOS平台为贯穿全文的实施载体。

前五部分递进展开设计蓝图——**确立范式**（为什么参照生产过程）→**建立对应**（制造经验如何翻译为运营方法）→**定义结构**（用什么数据支撑运营）→**设计工位**（人看到什么、怎样工作）→**端到端演示**（跟着一个构件走完全程）。第六部分提出**滚雪球法**——不要求任何前提条件，方法和数据在业务执行中自然累积，从一片雪花越滚越大，AI加速人的判断而非替代人。

核心洞见：把知识工作从手艺变成工艺。 工艺，是被工程化的手艺。落地，是让雪球自己滚起来。抓手，始终是构件——从一个构件开始构建，以构件的工艺路线为运行轨道，以构件的运行数据为完善依据。

用于后续91规范相关章节的参考。

目录

第一部分：确立范式 —— 为什么EOS参照生产过程构建

- [生产过程方法：术语界定](#)
- [一、认识工艺 —— 它是什么，如何运转](#)
 - [1.1 工艺的定义](#)
 - [1.2 工程化：把手艺变成系统](#)
 - [1.3 工与艺：能说清楚的和说不清楚的](#)
 - [1.4 过程、流程与工艺：同一个东西的三张面孔](#)
 - [1.5 工艺开发、任务策划、执行与监督](#)
 - [1.6 运行态与演化态：一次执行与持续改进](#)
- [二、论证 —— 为什么EOS需要生产过程](#)
 - [2.1 软件工程少了一层](#)
 - [2.2 其他范式为什么不行](#)
 - [2.3 知识工作比制造更需要工艺](#)
 - [2.4 工艺执行平台：系统运行工，支撑人做艺](#)
 - [2.5 创新也需要生产过程](#)

- 2.6 三类工作台全景

第二部分：建立对应 —— 制造经验如何变成EOS的开发方法

- 三、构件分类 —— 运营体系的组织单元
 - 3.1 三层分类：业务域 → 项目类型 → 构件类型
 - 3.2 识别方法：五步法
 - 3.3 构件分类是运营体系的骨架
 - 3.4 与制造业的对照
 - 3.5 从构件到工艺路线：完整链条
 - 3.6 关键概念：场景业务 —— 路线模板的项目实例
- 四、三个工艺对象 —— 工艺路线、工序卡片与流转卡
 - 4.1 一张图看懂三者的位置
 - 4.2 工艺路线 (Routing)
 - 4.3 工序卡片 (Operation Sheet)
 - 4.4 流转卡 (Traveler)
 - 4.5 三者如何协作
- 五、搭建映射 —— 工序、工作中心与工位如何对应到EOS
 - 5.1 工序不直接对应工位
 - 5.2 为什么需要工作中心这一层
 - 5.3 EOS的对应：工序→步骤，工作中心→角色
 - 5.4 作业指南不是工艺路线
 - 5.5 工序的判定标准
 - 5.6 一份文档的开发才是一道工序
 - 5.7 四个常见疑问
- 六、路线生命周期 —— 版本如何控制、路线如何设计
 - 6.1 版本的定义
 - 6.2 版本号规则
 - 6.3 路线与作业指南如何联动升级
 - 6.4 已启动的任务不受路线升级影响
 - 6.5 路线设计器：CAPP的EOS对应
 - 6.6 EOS路线设计器的布局
 - 6.7 设计器的入口与操作
 - 6.8 路线继承

- 6.9 路线设计器的操作协议

第三部分：定义结构 —— EOS用什么数据支撑运营

- 七、三层骨架 —— 主表、子表与孙表
 - 7.1 完整层次
 - 7.2 主表：构件文档清单
 - 7.3 子表：文档作业步骤清单
- 八、展开孙表 —— 每一步的五个问题与运行时产出
 - 8.1 孙表A：步骤资源清单——"我拿什么来做？"
 - 8.2 孙表B：步骤规则清单——"做得对不对？"
 - 8.3 孙表C：步骤检查清单——"做完了没有？"
 - 8.4 孙表D：步骤合规清单——"合不合外部规矩？"
 - 8.5 孙表E：步骤异常清单——"出问题了怎么办？"
 - 8.6 五张表的分工：谁来判定、怎么处理
 - 8.7 运行态产出：四张检查结果清单

第四部分：设计工位 —— 人在EOS里看到什么、怎样工作

- 九、系统在背后做什么 —— 四步流水线
 - 9.1 李四接到一个任务
 - 9.2 第一步·定位：找到该读的那一页
 - 9.3 第二步·解析：翻开作业指南
 - 9.4 第三步·上下文：装配手边的资源
 - 9.5 第四步·渲染：工位就绪
- 十、工位长什么样 —— 理解型与选用型步骤的页面
 - 10.1 Step 1 工位页面——理解型步骤
 - 10.2 Step 2 工位页面——选用型步骤（检索中）
 - 10.3 Step 2 工位页面——选用型步骤（已选定）
 - 10.4 Step 1 的完整操作流示例
- 十一、TAB菜单 —— 孙表数据如何变成操作界面
 - 11.1 菜单的每一项来自哪张孙表
 - 11.2 一次完整检查的详细过程
 - 11.3 文档进度是怎么算出来的
- 十二、辅助栏 —— 七种资源点开之后看到什么
 - 12.1 输入文档下钻

- 12.2 实体库检索面板激活
- 12.3 工作中心下钻
- 12.4 看板下钻
- 12.5 指标下钻
- 12.6 模板下钻
- 12.7 资源清单的三层下钻总表

第五部分：端到端演示 —— 跟着一个构件在EOS中走完全程

- 十三、全程记录 —— 电源模块从路线到交付
 - 13.1 角色表
 - 13.2 第一幕：赵总工编制路线
 - 13.3 第二幕：钱经理启动项目
 - 13.4 第三幕：张三开发第一份文档
 - 13.5 第四幕：李四开发第二份文档
 - 13.6 第五幕：后续工序推进
- 十四、回看 —— 执行者、管理者与每个人感知到了什么
 - 14.1 场景中各步骤的工位页面对比
 - 14.2 管理者的场景业务视图
 - 14.3 赵总工、钱经理与执行者们各自感知到了什么

第六部分：滚雪球法 —— 组织如何构建、运行与演化EOS

- 十五、启动阶段 —— 从 -1 到 0.1
 - 15.1 启动前自我诊断
 - 15.2 零条件启动：三条不需要任何人批准的路径
 - 15.3 第一个正式试点的执行手册
 - 15.4 试点验收：4项通过标准 + Go/No-Go/Adjust
- 十六、扩展阶段 —— 从 0.1 到 1
 - 16.1 扩展决策：什么时候从1个构件到1类构件
 - 16.2 Owner制度启动包
 - 16.3 第一次信任危机的处理流程
 - 16.4 惯性突破的四个锚点
- 十七、自治阶段 —— 从 1 到 N
 - 17.1 演化态启动的判断标准
 - 17.2 路线季度检视与修订流程

- 17.3 数据健康度季度自检
 - 17.4 组织治理结构的落地包
 - 17.5 五个阶梯的总览
 - 十八、总结
-

第一部分：确立范式 —— 为什么EOS参照生产过程构建

生产过程方法：术语界定

在进入正文之前，先界定本文标题中“生产过程方法”的确切含义——它指的是什么、不是什么、以及为什么用这个术语。

它是什么。 生产过程方法是制造业组织协作活动的完整结构——包含四个环节：

工艺开发 → 任务策划 → 工艺执行 → 监督评价
(设计怎么做) (这次谁做、什么时候做) (实际做、边做边查) (做得怎么样、怎么改进)

环节	制造业主语	回答的问题	EOS 对应
工艺开发	Process Planning / 工艺设计	"用什么方法做？"	赵总工：编制路线、配置作业指南、定义孙表
任务策划	Production Planning / 生产计划	"这次具体谁来做、什么时候？"	钱经理：创建业务启动指令、指定执行者、锁定路线版本
工艺执行	Production Execution / 生产执行	"按方法做，做得对不对？"	李四/张三/王五：工位操作、做→查→过、异常触发
监督评价	Production Monitoring / 生产监控	"实际和计划差了多少？归因是什么？"	钱经理：场景业务视图、检查结果汇总、阻塞项归因

这四个环节不是被设计出来的——它们是目的性协作在组织尺度上的逻辑必然。任何有目的的协作，都必须依次回答四个问题：用什么方法做？→ 这次具体怎么做？→ 实际去做。→ 做得怎么样？一个人做一根矛时，四个问题全在脑子里瞬间完成。一百个人协作时，它们必须外化到不同的角色承担 (§1.5 展开)。

把知识工作变成流水线，正是本文的主张。 这不是一个需要回避的词。制造业花了一百年把一个作坊里的手艺活变成了可工程化、可复用的生产线——知识工作今天面对的是同样的挑战。但“流水线”常被误读为“把人当机器”，这里做出区分：

	把人当机器的流水线	本文主张的生产过程
标准化什么	人的动作	方法的步骤和判定标准

	把人当机器的流水线	本文主张的生产过程
重复什么	人重复同一个动作	方法被不同的人重复使用
检验什么	检验产品是否符合规格	检验执行是否偏离方法、方法本身是否有效
人做什么	人被降级为执行单元	人的判断力被升级到更高层级——"工"的部分系统跑，"艺"的部分人判断
核心机制	消除人的差异	让方法的优劣不再取决于谁来做

两者的区别不在"有没有流水线"，而在**流水线承载的是什么**——是承载人的重复体力劳动（把人当机器的流水线），还是承载方法的工程化运转（把人解放出来做判断的流水线）。本文主张的是后者。§2.3 的论证——"知识工作比制造更需要工艺"——正是基于这个区分：知识工作如果不做工艺化，100% 都在艺里，十个项目经理十种管法，质量完全依赖个人。生产过程方法的目标不是消除判断力，而是**把低层级的判断力从人身上剥离、固化为系统可自动执行的方法资产，让人把判断力投入到更高层级的问题上**。

一、认识工艺 —— 它是什么，如何运转

1.1 工艺的定义

工艺 (Process / Manufacturing Process) 有三层递进的含义。

第一层：WHAT 与 HOW 的分离。 产品设计定义"是什么"（几何形状、公差、材料），工艺定义"怎么做"（用哪道工序、什么参数、什么工装）。同一个零件可以有不同的工艺路线——取决于设备、批量、成本策略。图纸不变，工艺可以完全不同。

第二层：方法的体系化。 工艺不是单个操作技巧，而是一个体系化的方法知识体：

工艺 = 工序序列 + 参数规范 + 资源约束 + 检验标准 + 异常处理

其中每一道工序都有确定的工艺参数（转速/进给/温度/时间）、声明的资源需求（设备/工装/刀具/量具）、内建的检验节点（规格+频率+判定标准）和预定义的异常处理路径。

第三层：从个体手艺到组织资产。 这是最关键的一层——工艺的本质是把生产方法从生产者身上剥离，固化为可传承、可审查、可优化的组织知识资产。

	手艺人模式	工艺模式
知识在哪	师傅脑子里	工艺文件中
传承方式	口传心授	文件体系
质量决定因素	谁做	按工艺做

	手艺人模式	工艺模式
人离开后	方法丢了	工艺还在
换人	重新摸索	按工艺文件执行

三层归结为一句话：**工艺，是被工程化的手艺。**

1.2 工程化：把手艺变成系统

§1.1 的结论——**工艺，是被工程化的手艺。** 这一节回答：什么是工程化？

定义

工程化是把依赖人的方法，转化为不依赖人的系统的过程。 它的对象不是产品，是方法本身——把“只有这个人做得好”变成“谁拿到这套方法都能做到及格线以上”。

工程化 = 外化 + 标准化 + 内建检验 + 系统化

把方法从 人脑子里 搬出来	给每一步 定义什么 叫“对”	把检验做进 方法里，不 依赖事后抽查	让方法脱离 人独立运转、 可传递、可追溯、 可改进
---------------------	----------------------	--------------------------	------------------------------------

不是一次性的“写文档”

工程化是一个持续的转化循环。手艺在脑子里时，方法是个黑箱——输入人、输出结果，中间靠什么，说不清楚。工程化分四步打开这个黑箱：

- 1. 外化**——把方法从人脑子里搬出来。“我知道该用 800 转”写下来变成“转速 800 ± 10 r/min”。这一步最难——很多手艺人能做但写不出来
- 2. 标准化**——给每一步定义什么叫“对”。不是“差不多就行”，而是“粗糙度 $\leq Ra1.6$ ，每 20 件抽 1 件用千分尺测”。工程化的标志性动作就是定义判定标准
- 3. 内建检验**——把检查做进方法本身，不依赖事后抽查。手艺模式是“做完了找人看一眼”，工程化之后检验是步骤的一部分——规则引擎在操作时按事件触发，边做边查
- 4. 系统化**——让方法脱离原执行者仍能运转。换一个人、换一个时间，拿到这套方法描述，产生的结果在同一个公差带内

七个特征

特征	含义	反例
有纪律	不是想怎么做就怎么做，有明确的规范约束	“随便写写就行”

特征	含义	反例
结构化	拆解为可管理的步骤、组件和判定节点	一块模糊的"把事情做了"
可重复	同一个人、同一方法、两次执行，结果在公差带内	"上次做得出来，这次不一定"
可验证	每一步有判定标准，做完能判断"对了没有"	"做完给师傅看一眼"
可传递	换一个人拿到方法描述，能做出一致的结果	"只有李四能做，王五得学三年"
可追溯	每一次执行有记录——谁、什么时候、做了什么、结果如何	"上次好像是谁做的来着"
可改进	方法本身可以被测量、被诊断、被优化	经验全在个人脑子里，组织不会从中学到任何东西

前三条是工程化的**门槛**——把模糊的事情说清楚、拆开、定规矩。中间三条是工程化的**质量**——方法脱离人也可靠运转。最后一条是工程化的**红利**——组织开始从方法中学习。

工程化的终点不是完美的方法

是可改进的方法。一套被工程化的方法可以被测量——哪个步骤的返修率高、哪个检查项经常卡住人——过程数据回流到方法设计者手里，方法本身成为迭代的对象。

与工和艺的关系

手艺是矿，工程化是提炼过程，工艺是成品。一个活动刚开始时完全靠艺——经验、手感、直觉。工程化把它逐步转化为工——可描述、可传递、可检验、可重复。剩下的、暂时无法被工程化的部分，仍然是艺——它在间隙里等待下一轮工程化的到来。

1.3 工与艺：能说清楚的和说不清楚的

"工艺"这个词本身包含了**工**和**艺**两种方法资产形态。

工是可以脱离执行者独立存在的方法描述。四个要件：可外化（能写下来）、可传递（别人看懂就能做）、可检验（有明确的判定标准）、可重复（不同人不同时间做出来结果一致）。

艺是无法脱离执行者独立存在的方法能力。那些还做不到可外化、可传递、可检验、可重复的方法要素。

工与艺不是两个东西，而是一条光谱：



全自动判定
系统跑一圈就能出结果
例：R1优先复用
"候选物料满足全部参数要求"
系统自动跑：比对参数→匹配→通过/不通过

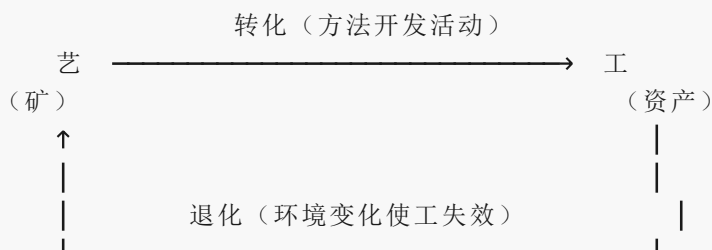
完全靠人判断
系统帮不上忙
例：这条路线的编排策略
对不对——"取决于行业趋势、
技术路线选择、客户偏好"
——全是说不清楚的东西
→ 方向选择层大量在这一端

← 孙表B的C1

← 孙表B的R4生命周期检查在这一带：
系统检测到"即将EOL"是工
评估"是否影响本项目"是艺

一个方法要素不是要么工要么艺——它的"工含量"和"艺含量"是连续的。R4生命周期检查：检测到EOL状态、标记⚡——系统自动完成的部分，是工；判断"本项目周期内是否受影响"——目前还说不清楚的部分，是艺。

工从艺炼：



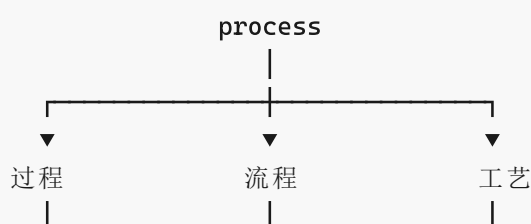
刚开始，一个活动完全靠艺——经验、手感、直觉。然后有人开始总结，描述越来越精确，变成可传递的工。环境变了、材料变了——原来有效的工艺失效，退回到艺，新的摸索开始，新的工再次被提炼。

艺不会消失、不应该消失。 消灭艺就是消灭未来的工。艺是组织认知的**前沿阵地**——一切还没有被说清楚的东西都在艺里。工是组织认知的**积淀层**——已经被说清楚、可反复使用的东西。

工艺开发是专门从事"艺→工"转化的工种。 赵总工的艺不是"知道怎么做路线"，而是"知道怎么把做路线的方法工化"。一个健康的组织，其工艺开发岗的真正产出不是工艺路线本身——而是"把艺转化为工的能力"的持续运转。

1.4 过程、流程与工艺：同一个东西的三张面孔

同一个 process，在运行时必然展开为三个维度：



时间轴上的展开 状态变化	步骤间的流转关系 先后顺序	做事的方法本身 参数/资源/检验/异常
-----------------	------------------	------------------------

这三个不是三个东西，而是一个东西的三个侧面。**流程是设计出来的，过程是发生出来的。**

	过程	流程	工艺
视角	实际发生的轨迹	设计的步骤序列	每一步的方法定义
时间方向	事后记录	事前设计	事前设计
可修改性	不可修改（记录）	可修改（走版本）	可修改（走版本）
可复用性	不可复用（一次性的）	可复用（模板）	可复用（模板）
制造载体	流转卡（Traveler）	工艺路线（Routing）	工序卡片（Operation Sheet）
EOS 对应	任务实例+检查结果清单	构件文档开发路线（主表）	作业指南（子表+五张孙表）

流程关心“关系”（节点之间的逻辑依赖），过程关心“状态”（节点在时间轴上的变化）。

这个区分解释了 EOS 中几个核心机制：

- “**锁定路线版本**”和“**异常回流更新**”可以同时存在——锁定的是**流程**（路线版本，不能做到一半改序列），回流的是**过程**（执行记录中的状态变化和变更传播）
- **检查清单是工艺，检查结果是过程**——检查清单（孙表C）事先设计、可复用；检查结果清单是“7月23日14:20本次检查C1通过、C2有警告”——这次执行的记录，不可复用
- **赵总工看到的是流程+工艺，钱经理看到的是过程**——一个是“方法对不对”，一个是“现在走到哪了”

1.5 工艺开发、任务策划、执行与监督

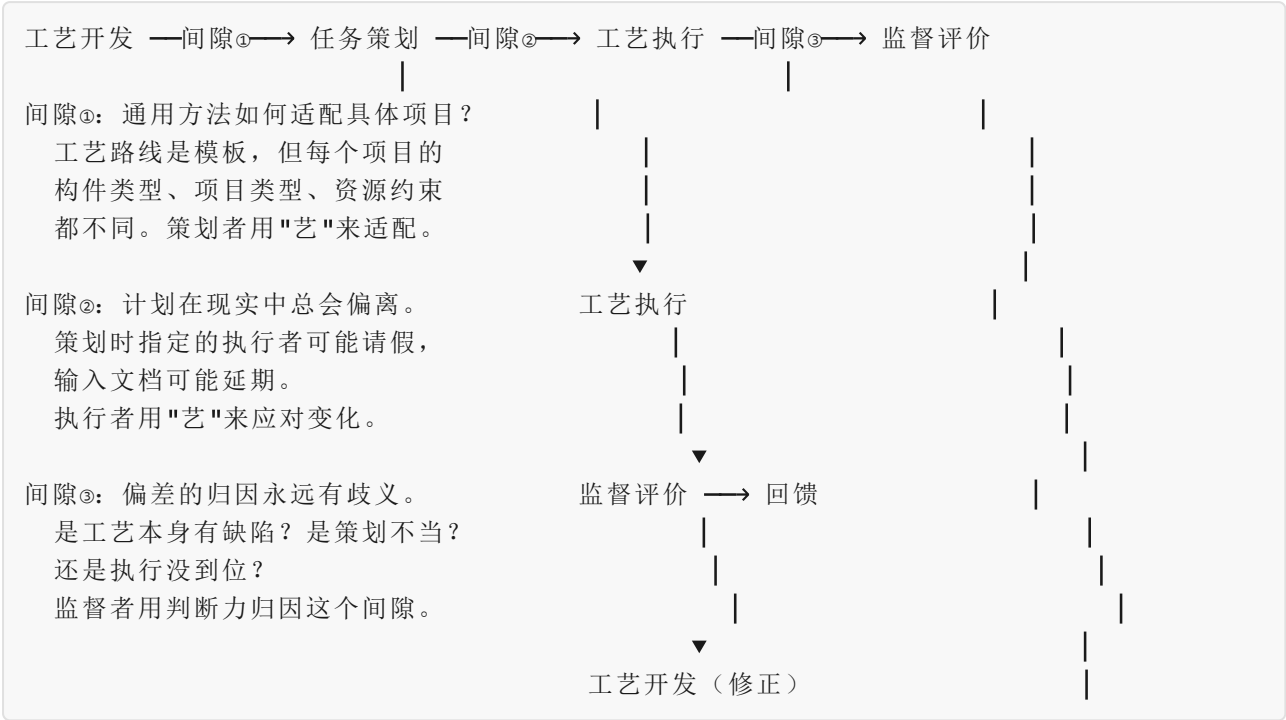
定义、结构、展开之后，下一个问题是：**工艺如何在组织中运转？**

答案是四个环节的分离——**工艺开发、基于工艺的任务策划、工艺执行、监督评价。**

这个四段式不是被设计出来的，而是目的性活动在组织尺度上的逻辑必然。任何一个有目的的协作，都必须依次回答四个问题：**用什么方法做？→ 这次具体怎么做（谁、什么时候、什么条件）？→ 实际去做。→ 做得怎么样？**一个人做一根矛时，这四个问题全在脑子里瞬间完成。一百个人协作时，它们必须外化到不同的角色承担。

其中，“基于工艺的任务策划”是容易被忽略但至关重要的一环。 工艺路线定义的是通用方法——“主轴加工需要五道工序”。但这次这批活——“7月10日启动，张三做工序1，李四做工序2，交期9月10日”——需要一个人把通用方法实例化为具体任务。制造业里这是计划员的职责（根据工艺路线创建生产订单、派发到工作中心），EOS里钱经理做的正是这件事（根据构件文档开发路线创建业务启动指令、指定执行者和期限、锁定路线版本）。

四个环节因三个不可消除的间隙而产生：



生产过程的智慧在于：它承认三个间隙无法被消除，但给每个间隙分配了明确的消化者。 间隙①由策划者的"艺"来消化（判断如何适配），间隙②由执行者的"艺"来消化（应对现实偏离），间隙③由监督者的判断力来消化（归因偏差），工艺开发者则在下一版工艺中把可固化的部分吸收进去。

四个环节各有不同的时间箭头：

环节	面向的时间方向	核心问题	EOS中的角色
工艺开发	面向 未来 ——在设计阶段定义"应该怎么做"	"这个方法对不对？"	赵总工（构件设计负责人）
任务策划	面向 未来到当下的转换 ——将通用方法实例化为具体任务	"这次谁来做、什么时候完成、用什么版本？"	钱经理（项目主管）
工艺执行	面向 当下 ——在操作中按方法做	"我在这一步该做什么？"	李四/张三/王五（文档开发者）
监督评价	面向 过去和未来之间 ——看偏差，驱动改进	"实际和计划差了多少？该改方法、改策划还是改执行？"	钱经理（项目主管）

1.6 运行态与演化态：一次执行与持续改进

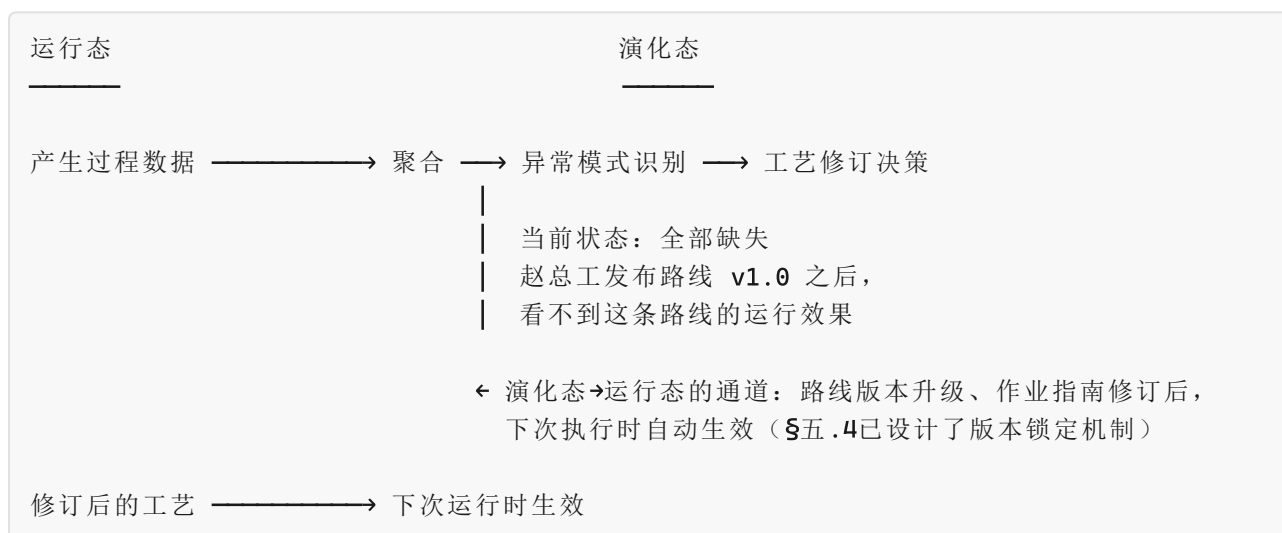
工艺的组织化运行中，系统里有两类性质不同的变化：

	运行态	演化态
什么东西在变	任务实例的状态	工艺资产本身
变化的范围	一次执行为界	跨多次执行累积
数据归宿	完成后归档（不可修改）	持续有效（版本管理）
谁在驱动	执行者 + 系统自动化	工艺开发者 + 数据信号
EOS中的对象	流转卡类比：检查结果清单、异常处理记录	工艺路线、作业指南、模板、异常路径定义

运行态是单次任务实例的执行——系统替人做的事（资源装配、状态监听、变更通知、规则校验）、人按工艺做的事（做→查→过）、以及执行中产生的一切过程数据（检查结果、异常记录）。

演化态是跨任务实例的工艺改进——过程数据的聚合和信号提取、工艺开发者的分析和决策、工艺资产的版本演化。

两层之间的关系构成完整的组织学习闭环：



当前设计中，运行态内部闭环已经详细展开（做→查→过→异常回流→TAB菜单→检查结果），但**运行态→演化态的回流通道的缺失**。赵总工在路线设计器里是一个“编辑者”，不是“演化的参与者”——他没有数据告诉他：这条路线被用了多少次、每道工序的平均周期、哪些异常路径被触发的频率最高、哪些作业指南的步骤经常卡住人。

工艺开发这个岗位本身也需要作业指南——他的“做→查→过”对象是一条路线，但目前缺少“查”（过程数据反馈）和“过”（效果追踪→判定是否需要修订）。

二、论证 —— 为什么EOS需要生产过程

§一回答了“工艺是什么”“工与艺什么关系”“过程/流程/工艺怎么展开”“生产过程怎么组织运转”“运行态与演化态怎么区分”。这一章回答：**为什么是 EOS 需要它，以及这对 EOS 平台设计意味着什么。**

2.1 软件工程少了一层

软件工程里的 process——无论是 CMMI、Agile 还是 RUP——都只定义**流程**（步骤序列和阶段门禁），不定义**工艺**（怎么做）和**过程**（执行记录）。

术语	实际内容	对应
Software Development Process	需求→设计→编码→测试→部署	流程——步骤序列和阶段门禁
Agile Process (Scrum)	Sprint Planning→Daily Standup→Review→Retro	流程——迭代节奏和角色分工
CMMI Process Area	需求管理/项目规划/配置管理	流程——该做什么、怎么评估成熟度

RUP 告诉你先做架构设计再做详细设计，但它不告诉你：一份合格的架构设计文档怎么写、用什么参数、查什么实体库、过什么校验规则、遇到异常怎么处理。**这些"过程"里没有工艺。**

制造业的 process 三层齐全，软件工程坍塌了：

制造业 process = 流程（路线）+ 工艺（卡片）+ 过程（流转卡）	← 三层齐全
软件工程 process = 流程（开发阶段定义）	← CMMI/Agile/RUP 全在这
工艺（怎么做）	← 缺失！落在个人身上
过程（执行记录）	← JIRA ticket/commit

坍塌的原因有三个：

- 知识工作是软的。** 制造业加工钢材，参数写错了出废品。知识工作的"工件"是文档和代码，"写得好不好"的边界模糊——工艺难以标准化，于是干脆不标准化了
- 工艺工程师的角色缺失。** 软件工程里谁在设计方法？架构师、Tech Lead——但他们的主要职责是做技术决策和带人，不是"设计开发方法"
- 手艺人的惯性。** 好程序员 = 有"艺"的人。把"艺"固化为"工"这件事在文化上有阻力——"写代码是创造性的，不能被工艺化"。这和两百年前手艺人抵制标准化生产的逻辑一模一样

EOS 在做的事，就是**把坍塌的工艺层重新撑起来**。作业指南不是 SOP（SOP 是流程工具），它是**工序卡片**——内建了参数（资源清单）、校验（规则+检查清单）、异常处理。它把散落在个人经验里的"怎么做"提取出来，放到它应该在的位置。

2.2 其他范式为什么不行

生产过程范式不是唯一的组织协作机制。把问题定义为"在组织尺度上让认知-行动-学习的循环闭合"，至少存在五个竞争者。它们各有自己的适用条件——也各有在企业的门里面跑不起来的原因。

范式	核心机制	适用条件	不适用企业运营的原因
价格机制 (市场)	价格信号传递信息+激励行动+反馈学习	交易成本低、产出可定价	企业内部的协调不能用每份文档招标
专业共同体	同行评议+执照准入+职业伦理	共同体边界清晰、训练周期长	企业运营跨专业边界，不是单一共同体
算法协调 (平台)	算法定义方法、记录执行、自动监督，三合一	任务高度标准化、不需专业判断	知识工作的每一步都需要判断
同行生产 (开源)	模块化切分+透明记录+PR审查+声誉激励	参与者自愿、产出可公开、高度技能	企业不是所有人自愿、不是所有产出可公开
委托指挥 (军事)	上级只给意图和边界，下级自主决定方法	人员高度训练、共享原则、深度信任	训练水平不一致、信任深度不够

价格机制。 哈耶克的论证——价格信号不需要任何人“设计方法”就完成了信息传递、激励行动、反馈学习。但它解决的是**企业之间的**协调问题，不是**企业内部**的。科斯的追问——既然价格机制这么有效，为什么会有企业？正因为企业内部用价格机制协调的交易成本太高。你不能每份文档都招标：“谁来写采购技术规范？报价。”

专业共同体。 医学、法律、学术——质量不是靠工艺路线保证的。靠的是同行评议、执照准入、职业伦理、十五年的训练把“工”刻进反射里。它确实闭合了认知-行动-学习循环。但前提是共同体的成员已经内化了方法，且协作不跨专业边界。企业运营恰恰是跨边界的——市场、研发、采购、售后不是同一个共同体。外科医生和麻醉师的协作已经超出了“共同体”的管辖范围；跨了十个专业领域的EOS运营更是如此。

算法协调。 Uber 不需要工艺工程师——算法同时做了“定义方法”（接单→到达→送达）、“记录执行”（GPS轨迹+时间戳）、“自动监督”（评分+投诉），三合一全在代码里。极其高效。但它只适用于任务高度标准化且不需要专业判断的场景——“写一份硬件设计方案”不是从A到B的移动，每一步都需要判断。

同行生产。 Linux、Wikipedia——没有工艺路线、没有工序卡片、没有监督岗。靠模块化切分、透明记录、PR审查、声誉激励，产出了高质量输出。但前提是参与者自愿、高度技能、产出可公开。企业不能假设所有人都自我驱动，不是所有任务都有人自愿领，不是所有产出都可以公开审查。

委托指挥。 普鲁士军队的模式——上级只交代意图和边界，下级自主决定方法。在高度不确定、信息不完整的战场上，这是唯一可行的范式。但它把“工艺”前置到了训练环节——士兵在战场上不需要工序卡片，是因为在训练场上已经把工步练成了本能。企业里训练水平不一致、信任深度不够，不具备这个前提。

企业运营不满足其他范式的前提条件，而生产过程范式的适用条件——任务可被方法化、方法可以文档化——恰好是企业运营中可以规模化建设的那一部分。 不是生产过程比别的范式更优越，而是别的范式在企业的门里面跑不起来。

2.3 知识工作比制造更需要工艺

直觉上，这个命题是反的。人们会说：制造生产加工的是物理对象——稳定、可测量、公差带明确——工艺化是自然的。知识工作的“原材料”和“产品”都是信息和判断——边界模糊、标准缺失、因人而异——你怎么工艺化？

恰恰因为不确定性更大、间隙更宽，知识工作更需要工艺结构。

回到 §1.3 的工与艺光谱。任何一项活动，总有一部分能做到可外化、可传递、可检验、可重复（工），另一部分暂时做不到（艺）。制造生产的“工含量”天然高——钢材硬度有国标，刀具寿命有曲线，转速的公差带可以实验测定。假设工含量能做到 80%，剩下 20% 的艺（材料微波动怎么调？异常怎么判？）——靠操作者的经验消化。组织仍然可控：80% 是标准化产出，20% 的偏差靠监督归因。

知识工作如果不做工艺化，工含量是 0%。100% 都在艺里。十个项目经理十种管法，十个架构师十个方案评审标准。偏差无法归因（是方法错了还是执行错了？——根本连方法都没有），质量无法追溯（上次怎么做的？——在那个项目经理的脑子里），经验无法积累（人走了方法就丢了）。

生产过程结构不是让知识工作“变得更僵硬”——它是让现在完全不可控的 100% 艺，降到 50% 艺 + 50% 工。 50% 的标准化校验由系统自动完成（资源装配、规则检查、检查清单判定），50% 的判断留给专业人的艺。制造生产花了 100 年才把工含量从 10% 提到 80%。知识工作才刚刚开始——但方向是一样的。

这和“知识工作是创造性的所以不能被工艺化”并不矛盾。回顾 §1.3：艺不会消失——它只是向上迁移。一百年前的工艺工程师纠结“转速该用 500 还是 600”，今天的工已经覆盖了这个问题，但他的艺用在了“产线瓶颈在哪里、怎么重新编排工序以提高 15% 的吞吐量”。知识工作的工艺化不是消灭判断力，而是**把低层级的判断力解放出来，投入到更高层级的问题上**。李四不需要纠结“这个物料供应商是否在 AVL 名录里”——那是 100% 的工，系统自动跑。他应该把判断力用在“即将 EOL 的 XH2.54-16P 在本项目周期内是否仍然适用”——那是工（EOL 预警）和艺（影响评估）的交叉地带，是他真正的价值所在。

2.4 工艺执行平台：系统运行工，支撑人做艺

回到 §1.3 的核心洞见：**工是系统能跑的，艺是人才能做的**。EOS 的角色由此有了精确的定义。

EOS 平台是一个工艺执行平台——它运行已被工化的部分，支撑尚未被工化的部分。 这不是一个模糊的“基础设施”概念，它有明确的边界：

系统做的事（工·自动执行）：

资源装配 — 遍历孙表A，逐条查询上下文，就绪的放上去，待就绪的标记 🕒 + 监听

规则校验 — 孙表B注册到校验引擎，操作时按事件触发，不通过标红 + 提示

人做的事（艺·系统支撑但由人判断）：

路线编排 — 为什么用7道工序而不是6道？赵总工判断。系统提供同类路线先例、构件

异常归因 — 偏差是工艺缺陷还是执行失误？钱经理判断。系统提供检查结果汇总、异常

检查清单 — 孙表C逐条自动判定通过/不通过，
附具体反馈到段落

战略方向 — 该不该做这个项目？
系统不替人做这个判断。
但它让信息可见——市场数据、竞品动态、资源

异常触发 — 孙表E监听条件满足→渲染操作入口，
自动创建子任务、关联段落、追踪回流

模板设计 — 作业指南的步骤类型和孙表配置
赵总工判断。系统提供执行数据反馈

这个分工不是静止的。演化态 (§一.6) 就是在做"艺→工"的转化——上一轮"人判断"的模式浮现了，下一轮就被工化，系统就能多跑一步。一百年前判"转速对不对"靠的是八级钳工的手感（艺），今天 SP C控制图自动报警（工），工艺工程师的艺用在了"这条产线的瓶颈在哪里"。EOS 的演化轨迹是一样的——今天李四判断"EOL 是否影响本项目"靠的是艺，未来这个判断逻辑被总结为一条规则注册到孙表B，系统又多了一条自动校验。

所以 EOS 不是"操作系统"（它不管理硬件和进程），也不是"文件系统"（它不是在被动存取数据）。它是一个工艺执行平台——一条不断向前推进的工/艺边界线。线的左边，系统全自动运行。线的右边，系统把信息装配好、把选项列出来、把结果追踪起来，支撑人做出更聪明的判断。演化态的使命，就是持续把这条线向右推进。

EOS 最终不以单一范式运行。工艺执行平台负责**可靠性**（工的部分），委托指挥负责**适应性**（艺的部分——异常边界中的人自主判断），同行生产负责**创造性**（知识的开放贡献和演化，让更多人的艺有机会被提炼为工）。三种范式叠加在同一个平台上，共享同一套工位骨架，但各自在最擅长的区域运转。

2.5 创新也需要生产过程

2.5.1 两个维度的乘法

生产过程的四个环节和创新活动的三个层级是正交的——**它们是乘法关系，不是加法或对立关系：**

生产过程维度（四个环节）：工艺开发 → 任务策划 → 工艺执行 → 监督评价
创新层级维度（三个层面）：选择新方向 → 开发新工艺 → 开发新产品

2.5.2 $4 \times 3 = 12$ 格

	工艺开发	任务策划	工艺执行
选择新方向 （做正确的事）	方向探索的方法 怎么发现机会？	将方向判断实例化 为具体调研任务 分配资源+期限	按方法探索方向 谁去调研？
开发新工艺 （搭建脚手架）	工艺开发的方法 怎么设计工序？ 怎么配置孙表？	将路线模板实例化 为具体构件项目 指定文档序列+执行者 钱经理创建业务启动指令	按方法开发工艺 赵总工在路线 设计器里操作
开发新产品	产品开发工艺	将作业指南实例化	按工艺开发产品

（用脚手架做事）	作业指南 五张孙表	为具体文档任务 分配执行者+期限 系统自动派发+通知	李四按做→查→过 写硬件设计方案
----------	--------------	----------------------------------	---------------------

2.5.3 关键推论

"生产过程"和"创新"不是对立的。 每一个创新层面都需要自己的四个环节——方向选择需要探索方法、也需要策划调研任务，工艺开发需要设计方法、也需要策划路线实例化，产品开发需要作业指南、也需要策划文档任务。创新的本质不是"不需要工艺"，而是"这一层的工艺还没被开发出来"。

当前覆盖状态——"任务策划"列是关键新知：

层面	工艺开发	任务策划	工艺执行	监督评价
方向选择层	空白	空白	空白	空白
工艺开发层	空白	空白	设计中（\$六·路线设计器）	空白——缺少过程数据回流
产品开发层	已有（作业指南配置）	已有（\$十二·第二幕·钱经理启动项目）	详细展开（后续各部分）	已有（检查结果清单+TAB菜单）

此矩阵的导航角色：12 格覆盖状态标注了当前设计的完成度和空白区。Part 25 聚焦**产品开发层**（最下面一行）——如何为一份文档开发配置作业指南、执行者如何按工艺操作、管理者如何监督。Part 6（滚雪球法）则是按层逐级爬升的路线图——\$十五十六（启动·扩展期）让产品开发层先跑通，\$十七（自治期·演化态启动）开始建设工艺开发层，方向选择层是数字化完成后的远景目标。后续各阶段开头将标注当前处于矩阵的哪个格子。

2.6 三类工作台全景

2.6.1 从十二格到三个工作台

12 个格子中，当前有明确需求的是**产品开发层全排 + 工艺开发层中排**。四个环节对应四个角色职能，但在 EOS 中它们由三个岗位（三个工作台）承载——钱经理的工作台覆盖了任务策划和监督评价两个环节：

工作台	使用者	覆盖的环节	覆盖的格子
执行者工作台	李四（文档开发者）	工艺执行	产品开发层·第3列
管理者工作台	钱经理（项目主管）	任务策划 + 监督评价	产品开发层·第2列 + 第4列
工艺开发者工作台	赵总工（构件设计负责人）	工艺开发（路线编辑）	工艺开发层·第1列

三类工作台共享同一套工位骨架（顶部上下文+正文+辅助栏+底部操作栏），承载不同层级的内容。钱经理的"管理者工作台"在不同阶段呈现不同界面——策划阶段是业务启动视图（创建指令、指定执行者、锁定路线版本），监督阶段是场景业务视图（进度条/阻塞项/检查结果汇总）。

2.6.2 文档覆盖状态

工作台	当前文档覆盖情况	主要缺口
执行者工作台	详细展开 （后续各部分）—— 四步流水线+七种资源下钻+辅助栏装配+TAB菜单	—
管理者工作台	策划侧已有 （§十三·第二幕·钱经理启动项目）+ 监督侧已有 （§十四·管理者的场景业务视图）	任务策划与监督评价的作业指南：如何判断路线适配、如何归因偏差
工艺开发者工作台	仅路线编辑工具 （§六·路线设计器）	过程数据回流、异常模式识别、效果追踪——即演化态闭环

第二部分：建立对应 —— 制造经验如何变成EOS的开发方法

第一部分回答了“为什么参照生产过程”。从这一部分开始，进入“怎么做”——**首先展开四个环节中的工艺开发环节**：构件分类（§三·工艺路线挂在什么上面）、制造业三个核心对象的映射（§四~五·工艺对象如何翻译为EOS概念）、路线生命周期管理（§六·版本如何控制、路线如何设计）。任务策划与监督评价在第五部分（端到端演示）和第六部分（组织落地）中展开。

三、构件分类 —— 运营体系的组织单元

3.1 三层分类：业务域 → 项目类型 → 构件类型

制造业中，一个物料号的工艺路线不是孤立存在的——它属于某个产品，该产品又属于某个产品线。运营体系中同样如此：工艺路线的锚点是构件类型，构件类型属于端到端项目，端到端项目属于业务域。

首先要澄清一个关键区别：**构件是项目输出产品的构件，不是项目分解结构（WBS）的构件**。构件来自产品架构的分解——一个设备产品的电气系统包含电源模块、电机驱动器等构件，这些是产品的组成部分。WBS是项目的管理分解——“需求分析”、“方案设计”、“采购”是项目的工作包，不是产品构件。运营体系的工艺路线挂在产品构件上，不是挂在WBS上。

端到端项目的严格定义：从需求提出到需求满足的完整业务活动。企业的端到端项目由三类相关方驱动——上级/监管方驱动经营类项目（如经营规划、合规管理），客户驱动产品类项目（如产品研发、产品生产、售后服务），内部员工驱动资源类项目（如HR管理、财务管理、IT服务）。

业务域	例：研发域
└ 端到端项目类型	例：设备产品开发项目
└ 构件类型	例：电源模块（自研件）
└ 构件实例	例：XYZ电机项目·电源模块

这三层对应BA（业务架构）的五级层级中的前三级：

层级	BA对应	决定了什么	举例
业务域	BA 1级	域级规范、域级实体库、域级角色类型	研发域
端到端项目类型	BA 2级（项目类型=产品类型）	路线模式偏好、项目级约束规则	设备产品开发项目

层级	BA对应	决定了什么	举例
构件类型	BA 3级（架构节点类型端到端业务）	构件工艺路线（哪些文档、什么顺序、由谁做）	电源模块（自研件）

为什么构件类型是运营体系的焦点？ 工艺路线挂在构件类型上——正如制造业的工艺路线挂在一个物料号上。同一个构件类型的所有实例共用同一条路线。构件类型的分类（自研件/外购件/标准件等）决定了路线的模式、实体库集合和约束规则集。**端到端项目类型是构件类型的容器，构件类型是工艺路线的锚点。** 识别和定义端到端项目类型与构件类型，是开发运营体系的第一块地基。

构件实现是广义的。 不是所有构件实现都是物理制造。设计态构件（如电源模块·自研件）的实现是从需求到经过验证的方案；维修态构件（如设备故障诊断）的实现是从问题到解决；采购态构件（如外购标准件）的实现是从规格到合格供方和物料；测试态构件（如产品验证）的实现是从测试计划到验证通过的结论。构件的“状态”决定了实现路径——也就决定了工艺路线的工序模式。

3.2 识别方法：从端到端项目到构件类型的五步法

构件类型不是凭空定义的——它们是从端到端项目的输出产品中逐层拆解出来的。这套方法在BA方法论中称为**输出产品架构锚定法**：以输出产品的架构节点类型为锚点，逐层推导出端到端业务和构件类型。以下是系统识别和定义构件类型的五步法。

第一步：识别并定义端到端项目。 端到端项目 = 从需求提出到需求满足的完整业务活动。判定标准：由三类相关方之一驱动（上级/监管方→经营类，客户→产品类，内部员工→资源类）、有明确的从启动到交付的生命周期、产出可独立验证的结果。典型例子：设备产品开发项目（客户驱动·产品类）、合规管理项目（上级驱动·经营类）、HR管理项目（内部员工驱动·资源类）。

第二步：推断端到端项目的输出产品。 每一个端到端项目都产出至少一个“输出产品”——实物产品、软件产品、服务、或方法工具。关键不是产品形态，是产品**对相关方的价值**。

设备产品开发项目的输出产品是“设备产品”——直观。但很多项目的输出产品不是实物，识别“系统”时会遇到困难。几个典型例子：

端到端项目	输出产品	产品的“系统”是什么
设备产品开发	设备产品	设备产品（实物系统）
故障产品处理	故障（被诊断和解决的故障）	故障——从现象到根因到解决方案到修复验证
供应商准入	合格供应商	供应商——从寻源到准入到绩效监控
员工招聘	到岗员工	员工——从需求到入职到适岗
合规管理	合规状态	合规——从法规识别到差距分析到整改验证

识别“系统”的方法：问自己“这个项目完成后，什么东西从无到有、或者从旧到新、或者从坏到好？” 答案就是输出产品。然后问“这个东西能分解成哪些组成部分？”——那就是产品架构。

以故障产品处理项目为例。完成后，一个故障从"未知"变成了"已知并解决"。输出产品是"故障"——但构成这个"系统"的构件是什么？不是处理流程的步骤（现象→根因→方案→修复），而是**被故障波及的所有实体**。一个设计缺陷引发的故障可能波及：故障产品、故障构件、设计态构件、复用该设计态的其他产品类型、在制构件、在制产品、成品、在役品、设计者、责任部门……每一个都是一类需要独立处理的实体。

在进入第三步之前，必须澄清三个关键概念：

1. 构件类型 ≠ 构件实例。 工艺路线挂在构件类型上，不是挂在实例上。100台受波及的在役品是100个实例，但同属"在役品"这一个构件类型——共用同一条工艺路线。实例数量的多寡不影响构件类型的定义。

2. 构件类型 ≠ 处理步骤。 "根因分析"是步骤，不是构件。"设计态构件"才是构件——它是一类实体（该构件的设计），它需要经历"问题确认→设计缺陷分析→修复方案→设计变更→验证"这个实现过程。构件是**实体类别**，不是**动作类别**。

3. 构件类型必须能承载实现它的产品数据序列。 一个实体从初始状态到目标状态的转化，必须能展开为一个有先后依赖、有质量判定标准的产品数据序列（=工艺路线）。如果一个候选实体无法展开出这样一个序列——比如"故障影响范围"只是处理过程中的一条信息，它本身没有独立的转化过程——那它不是构件类型，而是某个构件的属性或数据项。**构件类型是产品数据序列的容器，产品数据序列是构件类型得以成立的证据。**

构件的分解维度取决于项目的性质，检验标准是统一的：每个构件类型是否都需要一个独立的、与他人不同的实现过程？ 需要→独立构件类型。不需要→合并。设备产品开发按物理架构分解，故障处理按波及的实体关系链分解——维度不同，但逻辑相同。

第三步：推断输出产品的产品架构。 所有产品都有一个根节点——**系统**。系统向下逐层分解为构件。设备产品：系统（设备产品）→ 构件（电源模块、电机驱动器）。故障：系统（故障）→ 构件（故障产品、故障构件、设计态构件、在制构件、在役品……）。产品架构的末级节点就是构件类型的候选。

第四步：定义产品构件类型。 在产品架构的每个末级节点上，定义构件类型。关键属性：构件类别（自研/外购/委外/标准件——决定路线模式）、在架构中的位置（决定前序后续依赖）、实现路径（设计态/维修态/采购态/测试态……——决定工序模式）。电源模块是"电气系统"下的构件类型，类别"自研件"，实现路径为设计态（需求→方案→规格→验证）。

第五步：定义构件实现的产品数据序列。 明确每类构件从初始状态到目标状态需要产出的产品数据——文档类型、数据规格、质量标准、先后依赖。这些产品数据构成了工艺路线的骨架：**一份产品数据 = 一道工序，数据之间的依赖 = 工序之间的前序关系**。产品数据的形态取决于构件的实现路径——设计态构件是"需求→方案→规格→验证"，维修态构件是"问题→诊断→方案→修复→验证"。五步法走到这里，工艺路线的骨架已经自然浮现，不需要重新设计，只需要确认和细化。

五步法的输出物： 端到端项目清单 → 输出产品定义 → 产品架构分解 → 构件类型清单（含类别、位置、实现路径） → 每类构件的产品数据序列。这套输出物构成了运营体系的"BOM"——制造业

的BOM定义了需要加工哪些零件，这套分类体系定义了需要为哪些构件类型开发工艺路线。**每一个构件类型的产品数据序列，就是它的工艺路线的第一版草稿。**

不需要一次穷举。 第一个试点只需要走通试点构件所在的那条链——从它所属的端到端项目开始，向下拆解到该构件类型即可。其他构件类型在扩展阶段（§十七）逐步补全。

3.3 构件分类是运营体系的骨架

构件分类一旦建立，整个运营体系的组织就有了锚点：

- **工艺路线的归属**——每条路线挂在一个构件类型上，该类型的任何实例（任何项目中的该构件）都引用同一条路线
- **路线模板的组织**——模板按“项目类型→构件分类”组织，同一项目类型下的同类构件共享路线模板
- **Owner 的管辖范围**——域工艺师管辖一个业务域内的构件路线，路线Owner 管辖一条或多条具体路线
- **数据聚合的维度**——过程数据按构件类型聚合，可以比较同一构件类型在不同项目中的表现
- **滚雪球的路径**——从第一个试点构件开始，先扩展到同项目类型下的同类构件，再扩展到其他项目类型

3.4 与制造业的对照

制造业	运营体系
产品线	业务域
产品型号	端到端项目类型
物料号（零件/组件）	构件类型
BOM（用料清单）	产品架构（系统→子系统→构件）
工艺路线（Routing）	构件工艺路线
工序卡片（Operation Sheet）	作业指南
流转卡（Traveler）	任务实例

制造业以BOM为骨架编排工艺路线，运营体系以产品架构为骨架编排构件工艺路线。逻辑完全相同——只是加工的对象从钢材变成了知识。

3.5 从构件到工艺路线：完整链条

把前面的内容串起来——开发一个业务域的运营体系，本质上就是做五件事：

- ① 识别该业务域有哪些端到端项目类型
- ↓
- ② 识别每个端到端项目类型的输出产品及其架构

- ↓
- ③ 从产品架构中识别构件类型
- ↓
- ④ 为每个构件类型定义产品数据序列（= 工艺路线的骨架）
- ↓
- ⑤ 为每个产品数据开发作业指南（= 工序卡片·做→查→过）

步骤	产出	对应文档位置
①②③	构件类型清单（含类别、实现路径、在架构中的位置）	§三
④	构件工艺路线（工序序列+每道工序的产品数据规格）	§四~六（制造映射→路线管理）
⑤	作业指南（步骤+资源清单+规则清单+检查清单+合规清单+异常清单）	§七~八（数据结构→孙表）

这就是运营体系构建的完整方法论：以构件类型为抓手，从业务域一路推导到作业指南。 制造业的工艺体系是"产品→BOM→工艺路线→工序卡片"，运营体系的工艺体系是"端到端项目→产品架构→构件类型→产品数据序列→作业指南"——逻辑完全相同，只是加工的对象从钢材变成了知识。

后续 §四~八 将沿着这条链，从制造业的三个工艺对象开始，逐一展开运营体系的映射、数据结构、和工位设计。

3.6 关键概念：场景业务 —— 路线模板的项目实例

在进入 §四之前，先定义全文后续高频使用的一个术语。

场景业务（Scene Business） 是一个构件类型在某次具体项目中的一次执行实例。它是组织和管理视角的概念——把一个项目中的一组有先后依赖的文档归为一个"生产单元"来统一管理。

- 场景业务 = 构件类型 × 项目实例 × 路线版本（锁定）
- 例： "XYZ电机项目·电源模块" 是一个场景业务——
- 构件类型 = 电源模块（自研件）
- 项目实例 = XYZ电机项目（#BI-2026-0089）
- 路线版本 = RT-PWR-MODULE-001 v1.0（创建时锁定）

与相关概念的关系：

概念	层面	可复用性	说明
构件类型	产品架构层	跨项目复用	工艺路线的锚点，定义了"这类构件需要哪些文档"
构件文档开发路线	工艺设计层	跨项目复用（模板）	一个构件类型的通用文档序列定义，走版本管理

概念	层面	可复用性	说明
场景业务	管理实例层	不可复用（一次性的）	路线在某次项目中的绑定实例，锁定路线版本
任务实例	执行实例层	不可复用（一次性的）	场景业务中的单道工序（一份文档）分配给具体执行者

场景业务模板是场景业务的创建模板——策划者（钱经理）创建场景业务时，选定构件类型和路线版本，系统自动生成文档序列骨架。场景业务模板预配置了路线模式偏好、默认实体库集合、项目级约束规则。

为什么需要这一层：构件类型和路线是通用模板，但每次项目的执行者、截止日期、具体约束都不同。场景业务是通用模板与具体执行之间的**实例化层**——这正是 §1.5 中"任务策划"环节的核心产出：钱经理将通用路线适配为具体项目的文档序列，指定执行者和期限，锁定路线版本。

后续 Parts 3~5 中，"场景业务文档序列""场景业务进度""场景业务视图"等术语均基于此定义。

四、三个工艺对象 —— 工艺路线、工序卡片与流转卡

4.1 一张图看懂三者的位置

制造业中有三个核心的工艺对象，分别位于设计层、计划层和执行层：



工艺路线和工序卡片在设计层（PLM/CAPP系统），由工艺工程师创建和维护。流转卡在执行层（MES系统），由生产订单触发系统自动生成，由操作工在执行过程中逐步填写。

4.2 工艺路线 (Routing)

它是什么

工艺路线是一个物料号的主数据——定义了这个物料从毛坯到成品的完整加工路径。它绑定到一个确定的物料号，回答一个问题："把这个东西做出来，需要哪几道工序？"

物料号：MAT-SHAFT-001（主轴）

工艺路线：RT-MAIN-SHAFT-001 v3.2

工序10·下料

|

工作中心A

锯床 G4020

0.5h/批

工序20·车削

|

工作中心B

车床 C6140

3min/pc

工序30·热处理

|

外协供应商

热处理炉

4h/批

工序40·磨削

|

工作中心B

磨床 M1432

2min/pc

工序50·检

工艺路线头 (Routing Header)

routing_id:	RT-MAIN-SHAFT-001	
product_id:	MAT-SHAFT-001	← 绑定到一个物料号（一个零件/一个组件）
product_name:	"主轴"	
version:	"v3.2"	← 变更时走版本管理，不是覆盖旧版本
status:	"已发布"	← 状态：草稿 → 已发布 → 已过期
effective_from:	2026-01-01	← 生效日期
bom_id:	BOM-MAIN-SHAFT-001	← 关联用料清单（BOM）
batch_qty:	200	← 标准批量（影响工时计算和排程）
time_unit:	"min"	← 工时单位

核心属性：

- **绑定到一个物料号**——不同物料有不同的工艺路线。同一个物料可以有多条工艺路线（不同的生产策略），但同一时间只有一条是"当前有效"的
- **版本管理**——变更时产生新版本，旧版本保留。生产订单创建时锁定当时有效的版本
- **关联BOM**——工艺路线定义了"怎么做"，BOM定义了"用什么做"，两者通过product_id关联

工序序列 (Operations)

每道工序 (Operation) 是工艺路线的骨架节点：

op_seq:	20	← 工序号（10,20,30 ... 序号间隔允许将来插入新工
op_name:	"车削外圆"	← 工序名称
work_center_id:	"WC-B-03"	← 工作中心（不是具体工位！排程按工作中心）
resource_type:	"车床"	← 设备类型
resource_id:	"C6140"	← 设备型号

工时（决定排程和成本）：

setup_time: 15 min	← 换产调试时间
run_time: 3 min/pc	← 单件加工时间
queue_time: 30 min	← 排队等待时间
move_time: 10 min	← 从上道工序转运到本道工序的时间

工序序列的核心特征：顺序固定、每道工序有独立的工作中心、工时是标准定额、工序号用10的倍数允许插入新工序。

版本管理

RT-MAIN-SHAFT-001 v3.2 ← 当前有效版本，新工单引用此版本
RT-MAIN-SHAFT-001 v3.1 ← 历史版本
RT-MAIN-SHAFT-001 v3.0 ← 历史版本

生产订单 WO-2026-0610 引用 v3.1 ← 6月创建时引用了当时的版本
生产订单 WO-2026-0715 引用 v3.2 ← 7月创建时引用了最新版本

v3.1 → v3.2 的变更记录：

工序20 精车转速：600 → 800 r/min，粗糙度从 Ra3.2 → Ra1.6
工序30 从自制改为外协

工单锁定工艺路线版本。 工单创建时用的是哪个版本，执行时就一直用那个版本——"这批活做了一半，工艺不能突然变"。

4.3 工序卡片 (Operation Sheet)

它是什么

工序卡片是工艺路线中一道工序的完整展开。它是工艺路线的组成部分，不是独立的主数据对象。一道工序的卡片包含了工人在工位上需要的全部信息——工步、参数、物料、工装、检验标准、异常处理方法、作业指导。

以"工序20·车削外圆"为例：

工序卡片：车削外圆

所属工艺路线：RT-MAIN-SHAFT-001 v3.2

工序号：20

基本信息：

工序名称：车削外圆 / 工作中心：WC-B-03 / 工位3 / 设备：车床 C6140

操作者技能要求：中级车工

工步序列：

工步20.1·粗车外圆

内容：粗车外圆至 $\varnothing 52 \pm 0.5$ / 刀具：YT15 外圆车刀（粗车）

主轴转速：500 r/min / 进给量：0.3 mm/r / 切削深度：2 mm

量具：游标卡尺 0-150mm / 自检：每 20 件抽检 1 件

工步20.2·精车外圆

内容：精车外圆至 $\varnothing 50 \pm 0.1$ / 刀具：YT15 外圆车刀（精车）

主轴转速：800 r/min / 进给量：0.1 mm/r / 切削深度：0.5 mm
量具：千分尺 25-50mm / 自检：首件 + 每 20 件抽 1 件

投入物料：

工件：40Cr 棒料 Ø55，长度 120mm（来自上道工序），每台数量：1 件

消耗品：切削液（自动补给）/ 棉纱（每班 1 卷）

工装刀具（自动回冲 ERP）：YT15粗车刀片（1片/200件）/ YT15精车刀片（1片/500件）

检验要求：

检验项1·外径：规格值 Ø50，公差 +0.1/-0.1，检具千分尺25-50mm

抽检频率：首件+每20件抽1件，SPC控制：开

检验项2·粗糙度：规格值 Ra1.6，检具粗糙度样块，首件+末件

检验项3·圆度：规格值 ≤0.02，检具圆度仪，首件

异常处理：

E1 刀具磨损（粗糙度≥Ra3.2）

→ 换刀片→重检首件→对上一件到本次抽检之间的所有零件追溯复检→恢复继续生产

E2 尺寸超差（外径超出50±0.1，连续3件）

→ 停机→开出NCR→技术员判定（可返修：返修后重检 / 让步接收：技术负责人签字
/ 报废：隔离→标记→记录报废数量）→调整设备→重做首件→首件合格后方可继续

作业指导（Work Instructions）：

WI-020-01（文本·必须阅读确认）：

1. 装夹棒料，伸出长度≥80mm，用百分表找正外圆跳动≤0.02
2. 粗车外圆至Ø52±0.5，转速500 r/min，进给0.3 mm/r
3. 精车外圆至Ø50±0.1，转速800 r/min，进给0.1 mm/r
4. 用千分尺测量外径——首件+每20件抽1件
5. 用粗糙度样块比对——首件+末件

WI-020-02（图片·可选）：装夹示意图 [cdm://spec/shaft-turning-setup.png](#)

工时定额：

准终工时（换产调试）：15 min / 单件工时：3 min

200件的标准工时 = 15 + 3×200 = 615 min ≈ 10.25h

为什么工序卡片是工艺路线的组成部分

工序卡片不能脱离工艺路线独立存在。一道“车削外圆”工序离开了“主轴加工”这条工艺路线就没有意义——它没有独立的产品产出，没有独立的物料号，没有独立的生产订单引用。**工艺路线是骨架，工序卡片是血肉。**制造业中两者曾经是分离的物理纸张——这是纸质媒介的限制。数字化后这道墙不存在了。

4.4 流转卡 (Traveler)

它是什么

流转卡是工艺路线×生产订单的实例化产物。它是MES中的执行载体——跟随着一批物料在车间各工序之间流转，逐工序记录实际执行结果。如果工艺路线是“怎么做主轴的通用方法”，流转卡就是“这一批200根主轴这一次的实际执行记录”。

流转卡在生产过程中的完整演变

T0 7月16日 08:00 · 流转卡创建

流转卡 #TRAV-2026-0715-003

工单: WO-2026-0715-003 产品: 主轴 计划数量: 200件 交期: 7月25日

工艺路线: RT-MAIN-SHAFT-001 v3.2

工序10□→工序20□→工序30□→工序40□→工序50□

T1 7月16日 08:30 · 工序10 完成

张三在工位A扫码开工→实际开始08:05→实际完成09:20

实际工时75min (标准100min, 比标准快25min)

投料: 40Cr棒料Ø55, 批号LOT-MAT-0715-001, 70m

产出200件合格, 报废0件

→ 工序20解锁

T2 7月16日 10:00 · 工序20 进行中

李四在工位B/工位3扫码开工→MES工位终端加载工序卡片→渲染工位界面:

└─ 图档: [零件图:主轴] [装夹示意]

└─ 作业指导: 粗车→精车→自检

└─ 物料: [扫码消耗 棒料] [自动回冲 刀片]

└─ 检验记录区 (待填写)

└─ 异常触发按钮

... 加工进行中 ...

第60件时粗糙度Ra3.5超标→触发E1 (刀具磨损):

系统提示→李四确认→更换YT15精车刀片

→追溯复检: 上一抽检点(第40件)到当前(第60件)的20件全部复检

→检出2件粗糙度超标→返修→重做首件合格√→继续

异常记录写入流转卡: E1刀具磨损@第60件·换刀+2件返修→全部合格√

T3 7月16日 20:30 · 工序20 完成

实际工时setup12min+run630min=642min (标准615min, 略超27min)

产出198件合格+2件返修后合格, 报废0件

物料消耗: 棒料69.2m、粗车刀片1片、精车刀片2片 (含异常更换)

→MES回写ERP: 工时确认、物料回冲、工序完成确认→工序30解锁

T4 7月17日~20日 · 工序30~50 依次完成

工序30 (外协) →工序40 (磨削·王五) →工序50 (检验·赵六) →全部合格√

T5 7月20日 16:00 · 流转卡全部完成归档→产品入库→工单关闭

ERP成本核算: 物料消耗/工时/异常成本/报废成本→计入产品成本

流转卡的核心特征

模板数据和执行记录的合成体。 工艺路线的模板数据 (标准工时、标准物料用量、标准检验值) 和实际执行数据 (实际工时、实际消耗、实际检验值) 全部记录在同一张流转卡上——前者供对比分析, 后者供追溯。

完成后归档，不可修改。 和质量体系要求有关——产品出厂后如果出现问题，需要追溯到具体哪一批、哪道工序、哪个操作者、当时的检验值。流转卡是这条追溯链的核心载体。

4.5 三者如何协作

	工艺路线	工序卡片	流转卡
是什么	一个物料号的完整加工路径	工艺路线中一道工序的详细展开	一批产品的实际执行记录
存在位置	PLM/CAPP（设计层）	PLM/CAPP（设计层）	MES（执行层）
生命周期	主数据，长期有效	主数据，随工艺路线版本管理	业务数据，完成后归档
与物料号的关系	绑定到一个物料号	绑定到一道工序	绑定到一个生产批次
与生产订单的关系	被生产订单引用版本	被生产订单通过工艺路线间接引用	直接对应一张生产订单
数量	一个物料号一条（一个版本）	一条工艺路线多条	一张生产订单一张
变更时	产生新版本	随工艺路线一起变更	不改——执行记录不可变
典型字段	工序序列/工作中心/工时定额	工步/参数/刀具/量具/检验/异常	实际操作者/实际工时/检验值/异常记录
谁在用	工艺工程师（设计）、计划员（排程时读取工时）	工艺工程师（设计）、MES（渲染工位终端时读取）	操作工（报工）、班组长（审阅）、质检（确认）、ERP（成本核算）
EOS对应	构件文档开发路线	作业指南（做→查→过）	任务实例（工位视图）

五、搭建映射 —— 工序、工作中心与工位如何对应到EOS

5.1 工序不直接对应工位

一个常见误解是"一道工序=一个工位"。实际上是一对多，中间隔了一层：**工作中心**。

工序（Operation）	工作中心（Work Center）	工位（Workstation）
工艺路线上的一个节点 定义了"做什么"	一组同类工位的集合 定义了"在哪做"	操作者实际站的地方 设备/台位/操作者

工序 20·车削
→ 加工内容：车削外圆
→ 声明在"工作中心B"做

工作中心B·车床组
→ 拥有 3 台车床
→ 具备车削能力
→ 排程时按工作中心负荷排

WC-B 的 3 个工位：
├─ 工位1：车床
├─ 工位2：车床 C
└─ 工位3：车床
→ 实际派工时再指定

工序声明了它在哪个工作中心做，但不指定具体哪台设备。 排产按工作中心整体负荷排程，派工时根据工位实际空闲状态分派。

5.2 为什么需要工作中心这一层

原因1：同类设备的可替代性。 三台车床都能做车削外圆。工艺工程师关心"有没有车削能力"，不关心"具体哪台车床"。

原因2：工艺路线不受设备增减影响。 新买一台车床→加入工作中心B→产能增加。工艺路线不需要改——它只声明"这道工序在工作中心B做"。设备增减是产能管理的事，不是工艺设计的事。

原因3：排程和派工的分离。 排程（ERP/MES计划模块）看工作中心级："明天车床组能不能排下？"三个工位的负荷之和。派工（MES执行模块）看工位级："李四你今天在工位2上做这批。"

5.3 EOS的对应：工序→步骤，工作中心→角色

制造业

EOS

工序（Operation）

→ 车削外圆
→ 声明在工作中心B 做

作业指南中的一个步骤（工步）

→ 选定内容对象
→ 声明由某个角色类型执行

工作中心（Work Center）

→ 车床组·3台设备
→ 具备车削能力

角色类型 / 能力组

→ 采购工程师·3人
→ 具备物料选型能力

工位（Workstation）

→ WC-B·工位2 · 李四

执行者的个人工作台

→ 采购工程师·李四的当前任务

5.4 作业指南不是工艺路线

在之前的讨论中，我们把**作业指南**对应为**工艺路线**，把**步骤**对应为**工序**。回顾 Part 1 的框架：工艺路线 = 流程（步骤序列），工序卡片 = 工艺（每一步的方法），流转卡 = 过程（执行记录）。之前的映射把子表当成了主表——把"怎么做一份文档"当成了"产出哪些文档"。

制造业的工序10产出"下完料的棒料"，工序20产出"车好的轴"——每道工序有独立的半成品，有独立的检验点，可以交接给不同的人。EOS的Step1不产出独立的东西——它只是"理解了输入"，没有可交付的中间产物。Step14都是同一个人连续完成的，不出工作中心，没有独立交接。~~**Step14不是工序，它们是工步（Sub-operation）。**~~一份文档的开发作为整体才是一道工序。

5.5 工序的判定标准

判定标准	工序 (Operation)	工步 (Sub-operation)
有独立产出物	✓ "工序20产出车好的轴"	X "粗车"是中间状态, 不交付
有独立检验点	✓ 工序结束时统一检验	X 检验在工序结束时统一做
有工作中心交接	✓ "从工作中心B到外协供应商"	X 不出工作中心
可独立排程	✓ "工序20排在工序10之后、工序30之前"	X "粗车和精车在同一工位连续完成"
可独立分配操作者	✓ "工序20分配给李四"	X "粗车→精车→自检, 李四一口气做完"

用这个标准来判定EOS文档开发：一份文档的开发有独立产出物✓ ("采购技术规范")、有独立检验点✓ (评审/批准节点)、有人交接✓ (从"采购工程师"交到"技术评审人")、可独立排程✓ (排在工序2之后、工序4之前)、可独立分配操作者✓ (分配给李四)。而Step1~4不满足这些标准。**一份文档的开发才是工序。作业指南 (做→查→过) 是工序卡片, 里面的步骤是工步。**

5.6 一份文档的开发才是一道工序

制造业	EOS
产品 (一个物料号) 例: 主轴 工艺路线 (Routing) └─ 工序10·下料 └─ 工序20·车削 └─ 工序30·热处理 └─ 工序40·磨削 └─ 工序50·检验 每道工序 → 一个工作中心 工序20 → 工作中心B·车床组 工序卡片 (Operation Sheet) └─ 工步序列 - 工步20.1·粗车 - 工步20.2·精车 └─ 投入物料 └─ 工装/刀具/量具 └─ 检验要求 └─ 异常处理 工作中心 (Work Center) 流转卡 (Traveler)	构件 (一个设计态构件) 例: 电源模块 (自研件) 构件文档开发路线 └─ 工序1·用户需求整理 └─ 工序2·设计方案 └─ 工序3·采购技术规范 └─ 工序4·实现方案 └─ 工序5·测试报告 └─ 工序6·验收报告 每道工序 → 一个工作中心 工序3 → 采购工程师组 (3人) 作业指南 (单份文档的开发指南) └─ Step1·理解需求 (工步1) └─ Step2·选定内容对象 (工步2) └─ Step3·编写正文 (工步3) └─ Step4·自检与提交 (工步4) ← 这些是工步, 不是工序 能完成该文档类型开发的人员集合 任务实例 (路线×业务启动指令)

5.7 四个常见疑问

问题1：为什么不同文档类型的“做法”根本不同？ 因为它们是不同工序的工序卡片。车削的工序卡片和磨削的工序卡片完全不同，正如采购技术规范的作业指南和测试报告的作业指南完全不同。

问题2：构件的文档序列和场景业务是什么关系？ 构件文档开发路线（=工艺路线）定义了“这个构件需要产出哪些文档、什么顺序”。场景业务是组织和管理视角——把一组有先后依赖的文档归为一个“生产单元”。两者是从不同视角看同一件事。

问题3：作业指南为什么绑定到一份文档类型而不是一个文档序列？ 因为它是工序卡片——工序卡片天然绑定到一道工序，而不是整个工艺路线。工艺路线才绑定到文档序列。同一个文档类型出现在不同构件的路线中时，引用的是同一份作业指南。

问题4：做→查→过的“过”去哪里？ “过”在工步级=进入下一个步骤。“过”在工序级=文档流转到下—份文档或进入评审节点。

六、路线生命周期 —— 版本如何控制、路线如何设计

6.1 版本的定义

构件文档开发路线的版本是这个构件类型在特定时间点确定的文档序列定义的快照——哪些文档、什么顺序、每份文档用什么版本的作业指南、由哪个工作中心执行。

6.2 版本号规则

语义版本——主版本.次版本：

- **主版本递增** (v1→v2)：新增/删除必出文档、文档顺序变更、工作中心变更、构件类型的分类变更（自研件→外购件，整个路线模式变了）
- **次版本递增** (v2.0→v2.1)：某份文档的作业指南升级到新版本、新增可选文档、构件级引用变更、某份文档的前序依赖调整（但不改变整体顺序）

6.3 路线与作业指南如何联动升级

作业指南发布新版本→路线不自动升级→路线设计者评估新版本作业指南的变更对当前构件是否适用→适用则升级路线次版本→不适用则保持。和制造业一致——工艺路线中引用的工序卡片版本升级了，工艺工程师需要评估对当前产品是否适用。

6.4 已启动的任务不受路线升级影响

已创建的业务启动指令锁定创建时的路线版本。 路线后续升级不影响进行中的任务。和制造业“工单锁定工艺路线版本”的原因完全相同——不能让批活做了一半，工艺突然变了。

6.5 路线设计器：CAPP的EOS对应

工艺工程师坐在CAPP系统前，做三件事：**选定零件**（打开一个零件号，CAPP从PDM中读取它的设计模型）→**生成路线**（系统根据零件特征匹配已有工艺路线模板+工艺知识库规则，生成一份候选路线）→**调整发布**（工艺工程师增删改工序、调整参数、细化工序卡片，确认后发布为正式工艺路线）。整个过程，工艺工程师不画图、不定义产品结构——那些是设计工程师在CAD/PDM里做好的。CAPP只做一件事：基于已有的产品定义，编排加工路径。

6.6 EOS路线设计器的布局

用户是构件设计负责人（=工艺工程师）。工作台布局：

构件工艺路线设计器

构件：电源模块（自研件） 路线版本：v2.1（草稿） 基于：自研件路线模板 v1.0

路线结构（可编辑）	选中项详情
工序1·用户需求整理 └ 文档类型/作业指南 └ 工作中心 └	工序2·设计方案 的基本信息/作业指南/前序后续依赖 [变更文档类型] [变更工作中心] [上移] [下移] [删除] [在此插入新工序]
工序2·设计方案 ◀ 选中 工序3·采购技术规范 ...	
[在末尾添加工序]	

路线级参数：默认实体库/规范引用/约束规则集
作业指南生成状态：工序1√v1.2 工序2√v2.0 工序3△v0.9(待确认) ...
[保存草稿] [预览完整路线] [发布路线 v2.1]

6.7 设计器的入口与操作

三种入口：

入口	说明	适用场景
创成式	为已有构件设计路线：系统根据构件类型+项目类型+同类构件路线自动推导	已有构件的路线首次编制
新类型	为新建构件类型设计路线：基于构件类别的路线模板推导	全新的构件类型
变异式	从同类构件路线变异创建：复制已有路线，改差异部分	最常用——大多数构件不是全新类型

系统推导： 基于四层信息自动推导候选路线——构件类别（路线模板）+项目类型（模式偏好）+同类构件路线（先例）+已有PL5前序文档定义（上下文）。覆盖度约85%。设计者做的事：审阅

系统推导→调整差异→补充缺失→确认作业指南→发布路线。

调整操作：

- **变更文档类型**：系统推荐候选文档类型（按匹配度排序），选定后自动关联对应作业指南
- **插入新工序**：系统根据上下文（前后工序的类型）推荐工序模板
- **上移/下移**：系统检查前序依赖关系，违反依赖时阻止拖放
- **查看/确认作业指南**：右区直接预览，可补充缺失的填写指引、检查项等

6.8 路线继承

路线可继承自平台级或产品类模板。模板升级→所有继承路线的设计者收到通知→自行评估是否采纳。

6.9 路线设计器的操作协议

§6.5~6.8 描述了路线设计器的 UI 布局和入口。本节定义设计器背后的**操作协议**——设计者与系统之间的交互契约：系统推导什么、设计者确认什么、什么情况下走变异式而非创成式。这是 Part 1 §1.5 中"工艺开发环节"在路线编制场景中的具体化。

6.9.1 系统推导与人工确认的分工

路线设计器的核心设计原则：**系统推导候选，人工确认差异**。系统不做最终决策，但它应该把设计者的工作从"从零设计"压缩为"审阅+调整"。

系统负责（自动化·工）	设计者负责（判断·艺）
① 加载构件上下文（类型/类别/前序定义）	④ 判断系统推导是否适用当前构件
② 匹配路线模板（按构件类别+项目类型）	⑤ 调整差异（增删工序、变更文档类型）
③ 推荐候选工序序列（基于同类构件先例）	⑥ 确认作业指南版本适配
自动填充：文档类型/工作中心/前序依赖	⑦ 补充系统无法推导的工序（如法规要求的特殊文
自动关联：作业指南（最新已发布版本）	⑧ 判断作业指南的填写指引是否需要补充
自动检查：前序依赖无循环/工作中心存在	

为什么系统不自动发布路线：路线是组织资产，影响所有引用它的场景业务。系统推导的结果需要经过设计者的人工确认——不是系统能力不足，而是这属于 §1.5 中"间隙①"的范畴：通用方法如何适配具体构件？这个判断只能由人做。

6.9.2 三种入口的适用判定

入口	触发条件	系统行为	设计者工作量
创成式	构件类型已定义，同类构件≥3条已有路线	取同类路线为样本，差异分析→候选路线	约15%差异需人工调整
变异式	同类构件已有路线，差异较小	复制源路线，标记差异字段待确认	约5-10%差异需人工调整

入口	触发条件	系统行为	设计者工作量
新类型	构件类别无先例	基于构件类别的路线模板推导框架	约40-50%需人工补充

入口选择的判定逻辑（系统自动推荐，设计者可覆盖）：

- ① 同PL3类型下是否有已发布路线？
- └ 是，≥3条 → 推荐创成式（样本充足，推导精度高）
 - └ 是，1~2条 → 推荐变异式（从最近发布的那条复制）
 - └ 否 → ②
 - └ 同PL2项目类型下是否有已发布路线？
 - └ 是 → 推荐创成式（跨构件类型参考）
 - └ 否 → 新类型（从路线模板开始）

6.9.3 作业指南版本的匹配与确认

路线设计器自动为每道工序关联作业指南，匹配规则：

- 按文档类型查找对应作业指南的最新**已发布**版本
- 如果作业指南存在多个版本，列出最近3个版本及其变更摘要，默认选定最新已发布版本
- 设计者确认：当前构件是否需要最新版本的全部变更？ → 是：确认 → 否：选择旧版本或等待作业指南修订
- 作业指南状态为"草稿"或"待确认"时，系统标记△并提示设计者跟进

为什么作业指南不自动升级（§6.3 的逻辑基础）：作业指南的新版本可能包含对特定场景的优化（如新增了某类物料的选型规则），但不一定适用于所有引用它的构件类型。设计者需要逐条判断。

6.9.4 路线发布前的系统自动检查

设计者点击[发布]时，系统执行以下门禁检查（全部通过后方可发布）：

检查项	判定逻辑	不通过时
工序完整性	至少1道工序，所有工序文档类型+工作中心非空	阻止发布，标注缺失项
前序依赖无循环	拓扑排序检查工序序列无循环依赖	阻止发布，标注循环路径
作业指南确认	所有工序关联的作业指南均已确认（非"待确认"状态）	阻止发布，列出未确认项
工作中心存在	所有声明的工作中心在系统中存在	警告（允许发布，但标注风险）
路线头信息完整	构件类型/类别/实现路径均已填写	阻止发布

第三部分：定义结构 —— EOS用什么数据支撑运营

第二部分定义了工艺对象及其 EOS 对应物，§六介绍了路线版本管理与设计器。这一部分进入数据结构层——路线设计器产出的路线、每条路线引用的作业指南、每份作业指南的五张孙表，在系统中以什么结构存储和运转。这也是 Part 1 中“流程（主表）→工艺（子表+孙表）→过程（检查结果清单）”三层对应的数据落地。**四环节视角**：主表和子表是工艺开发环节的产出（赵总工设计路线和作业指南时定义的结构），孙表是工艺执行环节的运行时数据来源（李四做工位操作时系统解析和校验的对象），检查结果清单是监督评价环节的数据基础（钱经理看进度和归因偏差的原始材料）。

七、三层骨架 —— 主表、子表与孙表

7.1 完整层次

主业务数据：构件文档清单（工艺路线·每个构件独有）

每行 = 一道工序 = 一份文档类型

子业务数据：文档作业步骤清单（作业指南·工序卡片·可跨构件复用）

每行 = 一个步骤 = 做→查→过

孙业务数据（一组清单，每张结构不同，挂在子表的每行下面）：

- └─ 步骤资源清单 ─ 需要什么（装配用）
- └─ 步骤规则清单 ─ 做得对不对（校验用）
- └─ 步骤检查清单 ─ 做完了没有（步骤出口闸门）
- └─ 步骤合规清单 ─ 合不合外部规矩（人判断）
- └─ 步骤异常清单 ─ 触发条件+动作+回流（决策用）

7.2 主表：构件文档清单（=工艺路线）

```
{
  "route_id": "RT-PWR-MODULE-001",
  "version": "v2.1",
  "component": {"pl3_type": "自研件", "pl4_wbs": "电源模块"},
  "operations": [
    {
      "seq": 1, "doc_type_id": "DOC-REQ-COMPILE",
      "work_guide_id": "DG-REQ-COMPILE-001", "work_guide_version": "v1.2",
      "work_center": "需求分析组", "required": true, "predecessors": []
    },
    {
      "seq": 2, "doc_type_id": "DOC-TECH-DESIGN",
      "work_guide_id": "DG-TECH-DESIGN-001", "work_guide_version": "v2.0",
```

```

    "work_center": "设计工程师组", "required": true, "predecessors": [1]
  },
  {
    "seq": 3, "doc_type_id": "DOC-TECH-SPEC",
    "work_guide_id": "DG-TECH-SPEC-001", "work_guide_version": "v2.1",
    "work_center": "采购工程师组", "required": true, "predecessors": [1, 2]
  }
],
"default_entity_libs": ["MAT", "REUSE", "AVL"],
"default_regulations": ["通用设计规范 v1.0", "产品域设计标准 v2.1"]
}

```

7.3 子表：文档作业步骤清单 (=作业指南·工序卡片)

```

{
  "work_guide_id": "DG-TECH-SPEC-001",
  "version": "v2.1",
  "doc_type_id": "DOC-TECH-SPEC",
  "steps": [
    {
      "seq": 1, "name": "理解需求", "type": "理解型",
      "do": "打开输入文档→逐份阅读关键章节→标记矛盾点和注意事项",
      "check": ["C1.1 所有输入已确认", "C1.2 版本已检查通过", "C1.3 矛盾点已标记"],
      "pass": "→ Step 2",
      "grand_children": {
        "resources": [...], "rules": [...], "checklist": [...], "compliance": [...],
      }
    },
    {
      "seq": 2, "name": "选定内容对象", "type": "选用型",
      "do": "在正文字段描述需求→检索声明的实体库→候选比对→验证约束→选定写入",
      "check": ["C2.1 实体引用字段已选定", "C2.2 约束规则全通过", "C2.3 阻断异常全触发"],
      "pass": "→ Step 3 (待就绪不阻断)",
      "grand_children": {
        "resources": [...], "rules": [...], "checklist": [...], "compliance": [...],
      }
    }
  ]
}

```

八、展开孙表 —— 每一步的五个问题与运行时产出

孙表是子表（作业指南）的展开——子表的每行下面挂着五张结构不同的孙表。它们分别回答了执行者在做这一步时的五个问题。§8.7 则描述这些孙表在运行态中产生的检查结果数据。

8.1 孙表A：步骤资源清单——“我拿什么来做？”

每行是一个"可装配的东西"。平台遍历这张表，逐行在运行时上下文中查询实例，已就绪的直接装配到辅助栏。

Step2·选定内容对象

序号	资源类别	资源标识	是否必须	获取/检索方式
1	输入文档	用户需求整理（角色:输入）	必须	查场景业务文档序列·类型匹
2	输入文档	接口定义（角色:输入）	必须	查场景业务文档序列
3	输入文档	合规要求（角色:合规）	必须	查场景业务文档序列
4	实体库	物料库(MAT)	必须	候选来源·检索字段:型号/参数/分 /供应商/认证/生命周期/价格/交期
5	实体库	复用构件库(REUSE)	必须	候选来源·检索字段:构件名/功能/ /接口/成熟度/引用次数
6	实体库	合格供应商名录(AVL)	必须	约束来源·选定物料后校验（非候
7	模板	采购技术规范模板 v2	可选	直接引用·模板库
8	参考文档	历史设计文档库	可选	按关键词检索·文档类型/业务

8.2 孙表B：步骤规则清单——"做得对不对？"

每行是一条校验规则——平台注册到校验引擎里。规则在操作时按事件触发，通过则静默，不通过则正文标红并附提示信息。阻断型不通过还会触发对应的异常路径入口。

规则和检查清单的区别：规则是"边做边查"（选定一个物料→立刻跑R1R4），流式的、事件驱动的。检查清单是"做完了一起查"（步骤结束时统一判定C1C4），快照式的、步骤出口闸门。

Step2·选定内容对象

编号	规则描述	触发条件	判定逻辑	阻断级
R1	优先复用	候选物料满足全部参数要求	直接选用（推荐不强制）	
R2	三家比价	成本占比>5%或单件价值>¥500	候选供应商≥3家	阻断
R3	供应商在册	选定物料后（所有物料都触发）	物料.供应商∈AVL	阻
R4	生命周期检查	选定物料后（所有物料都触发）	已停供→阻断 即将EOL→警告 量产/成	

每条规则有四要素：

- **触发条件**——判断"这个事件我该不该响应"
- **判定逻辑**——该响应时怎么判定通过/不通过
- **阻断级别**——不通过时的严重程度（阻断/警告/推荐）
- **不通过→异常**——阻断型不通过时指向孙表E的哪条异常路径

8.3 孙表C：步骤检查清单——"做完了没有？"

完成标准——系统在步骤检查时统一逐条判定。多数条目自动判定，少数标记【需人工确认】。

Step2·选定内容对象

编号	检查项	判定逻辑	判定
C1	所有实体引用字段均已选定	扫描正文字段类型="实体引用"，全非空且引用目标存在	
C2	已选定全部通过约束规则	逐条重跑R1~R4，阻断型必须全部通过，警告不阻断	
C3	阻断型异常全部已触发	所有已触发的阻断型异常的action已执行，待就绪不阻断	
C4	关键器件选型已与项目主管确认	对标记为"关键器件"的物料，逐条检查确认状态	

8.4 孙表D：步骤合规清单——"合不合外部规矩？"

必须遵循的外部法规/标准/制度。人判断，系统不跑自动校验——但可辅助查询认证状态。

Step2·选定内容对象

编号	合规要求	适用判定	依据
L1	供应商须通过ISO9001认证	选定物料时检查供应商认证资质·系统可辅助	公
L2	电源模块须满足RoHS 2.0	选定物料时检查认证标记·系统可辅助查询	欧盟
L3	新物料须完成DFMEA分析	触发E2时·系统无法自动判定·需人工确认	产品

8.5 孙表E：步骤异常清单——"出问题了怎么办？"

触发条件满足时，平台在辅助栏中渲染对应的操作入口。异常的触发来源有两种：规则不通过（E1/E3/E4由对应规则不通过触发）和检索结果事件（E2由实体库检索结果=0触发）。

Step2·选定内容对象

编号	异常描述	触发条件	触发动作	备注
E1	候选不足	R2不通过(候选供应商<3)	扩大检索→仍不足→记录原因+标记裁决	
E2	物料缺失	检索结果=0	创建新物料寻源任务·描述自动提取·关联段落 不阻断步骤流转	#SRC-x
E3	供应商不在册	R3不通过(供应商∉AVL)	创建供应商准入任务·关联到文档段落	
E4	物料EOL	R4不通过·已停供/即将EOL	已停供→阻断选用 即将EOL→警告+替代入口	

触发后系统按声明在辅助栏中渲染操作入口——执行者不需要去别处找"怎么发起寻源"，入口就在眼前。点击后系统自动创建子任务、关联段落、标记文档、追踪回流。

8.6 五张表的分工：谁来判定、怎么处理

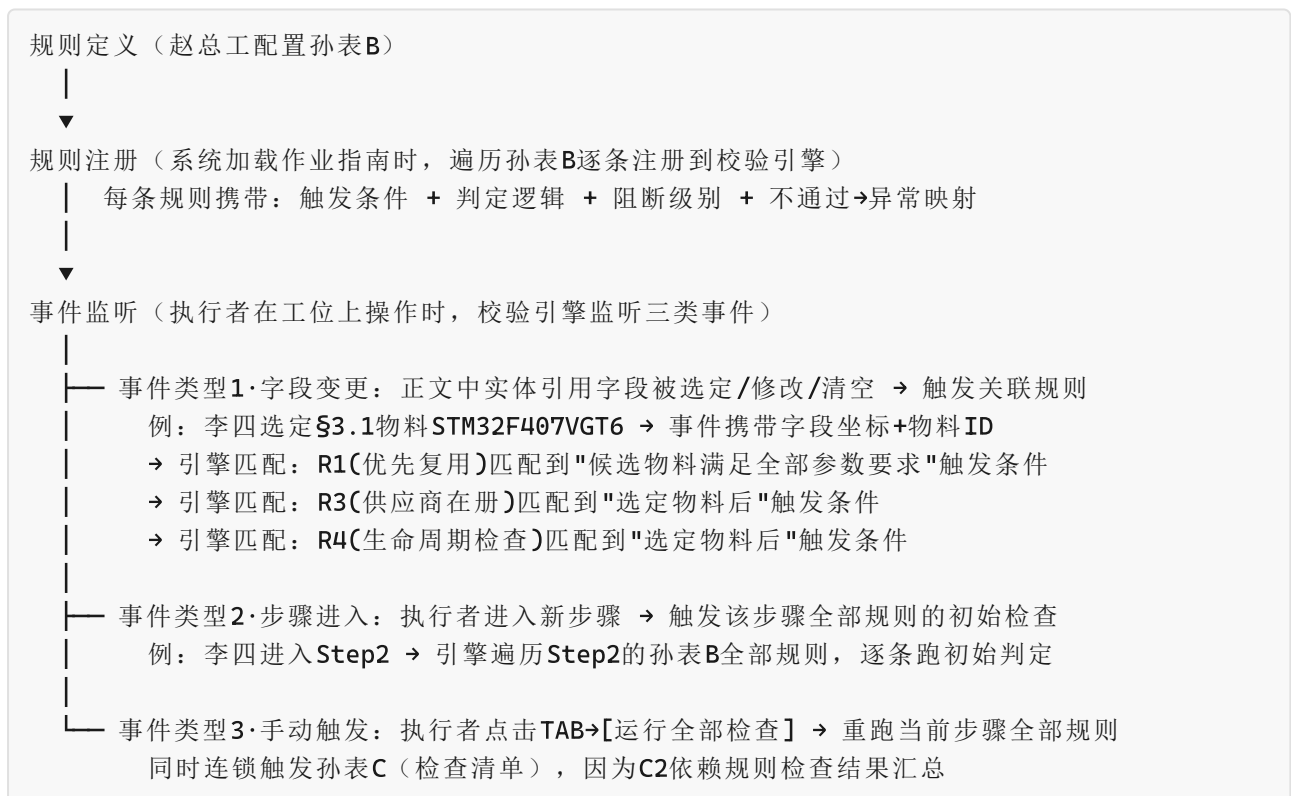
	平台处理方式	工位上的表现	判定主体
资源清单（孙表A）	装配 已就绪→放上去 待就绪→标记 🕒 +监听	辅助栏中可见可交互	系统
规则清单（孙表B）	注册到校验引擎 操作时按事件触发	正文标红或静默 不通过→标红+提示	系统
检查清单（孙表C）	注册为完成标准 步骤检查时统一跑	辅助栏中打勾列表 未通过→附具体反馈	系统为主 个别人工确认
合规清单（孙表D）	不注册不跑校验 只做提示和记录	辅助栏中列出 人逐条确认后勾选	人 系统可辅助查
异常清单（孙表E）	注册为事件监听器 监听规则不通过/检索结果	条件触发时出现操作入口 [发起新物料寻源]等	系统+人

五张表分别回答了执行者在这一步的五个问题： 资源清单→我拿什么来做？ 规则清单→做得对不对？ 检查清单→做完了没有？ 合规清单→有没有违反外部规矩？ 异常清单→出问题了怎么办？

8.6.1 规则引擎执行模型

孙表B的规则不是静态检查项——它们在运行时被注册到校验引擎中，按事件驱动的方式触发。理解这个执行模型是理解 Part 4 中"正文标红""辅助栏阻断提示""TAB菜单状态标记"等 UI 行为的前提。

规则的生命周期：



单条规则的判定流程：

事件触发

- ① 匹配触发条件
 - 不匹配 → 静默跳过（如R2"成本占比>5%或单价>¥500"，两个物料均<¥500 → 不触发）
 - 匹配 → 进入判定
- ② 执行判定逻辑
 - 输入：事件携带的数据（字段坐标、物料ID、上下文）
 - 输出：通过 / 不通过
- ③ 按阻断级别分流
 - 推荐（R1）：不通过 → 正文段落标记蓝色提示，不阻断任何流转
 - 警告（R4·即将EOL）：不通过 → 正文段落标记⚠黄色，辅助栏完成标准区标注"警告不
 - 阻断（R2/R3/R4·已停供）：不通过 → 正文段落标红+inline提示信息
 - 同时触发孙表E中声明的异常路径入口

规则检查结果的数据结构（单条）：

```
{
  "check_id": "RULE-CHK-2026-0723-001",
  "timestamp": "2026-07-23T14:20:00",
  "step_id": "STEP2",
  "rule_id": "R4",
  "rule_desc": "生命周期检查",
  "trigger": "字段变更",
  "target": {"field": "$3.3", "entity_id": "MAT-CON-00521"},
  "result": "warning",
  "detail": "XH2.54-16P 即将EOL，最后采购日期 2026-12-31",
  "action": "查找替代 / 评估影响 / 确认保留",
  "resolved": false,
  "resolved_by": null,
  "resolved_at": null
}
```

与孙表C的关系：孙表C（检查清单）不独立运行——C2"已选定全部通过约束规则"是对孙表B全部规则最新结果的**汇总判定**。规则引擎每次跑完当前步骤所有规则后，自动更新C2的汇总结果。C1/C3/C4各自独立运行。

8.6.2 异常生命周期状态机

孙表E中的每条异常路径定义了触发条件和触发动作。但触发只是异常生命的起点。一个完整的异常从触发到关闭经历五个状态，这个状态机横跨了执行者（李四）、异常接收者（周八）、和受影响的文档（硬件设计方案 §3.2）。

触发（Triggered）

- 触发条件满足（规则不通过 或 检索结果=0）
- 系统自动：创建异常记录 + 执行触发动作 + 在正文段落上标记状态
- 正文标记：⚡（阻断）或 ⏳（待就绪）

已分派 (Assigned)

- | 系统按孙表E声明的异常接收方类型自动分派 (或人工指定)
- | 被分派者收到通知 + 任务列表中出现异常处理任务
- | 正文标记更新: ⚡ → "处理中 #SRC-xxxx"



处理中 (In Progress)

- | 接收者执行处理动作 (寻源、准入、替代评估.....)
- | 处理过程产生自己的数据 (寻源报告、准入审核记录.....)
- | 这些数据通过异常ID与原始文档段落双向关联



已完成 (Resolved)

- | 接收者标记处理完成 → 系统检查回流条件是否满足
- | 回流条件满足 → 通知原始执行者: "§3.2的异常已处理完成"
- | 回流条件不满足 → 状态回退到"处理中" (如寻源找到了物料但未入库)



已回流 (Closed)

- | 原始执行者回到文档, 重新执行受影响的步骤
- | 例: 李四打开 §3.2 → 检索到新物料 LMR14030 → 选定 → 重跑规则 → 全部通过
- | 异常记录归档, 正文标记清除 (或变为"已解决✓")
- | 异常处理全过程记录写入异常处理记录清单

关键设计决策:

- **不阻断原则:** 异常触发后, 当前步骤仍可流转。执行者不需要等待异常解决才能进入下一步。正文段落标记"待就绪 ⏳"但不灰化整个步骤 (区别于"输入文档未就绪"的灰化处理)。这是知识工作与制造业的重要差异——工序间依赖是信息就绪约束而非物理因果约束, 允许并行推进。
- **回流协议:** 异常关闭后, 系统主动通知原始执行者 (不是执行者自己去查)。执行者回到受影响段落重新操作后, 异常才算真正关闭。如果执行者不响应回流通知, 异常保持在"已完成"状态, 管理者视图中的阻塞项继续显示。
- **异常接收方的确定:** 孙表E声明异常路径时指定接收方类型 (如"寻源工程师"), 系统在运行时按工作中心找人 (如有多个候选, 由管理者指定)。

异常处理记录的数据结构 (单条):

```
{
  "exception_id": "EXC-2026-0723-002",
  "exception_type": "E2",
  "exception_desc": "物料缺失——电源模块12V→3.3V/5V",
  "trigger_source": "检索结果=0",
  "trigger_step": "STEP2",
  "trigger_field": "§3.2",
  "doc_id": "DOC-0715-005",
  "status": "in_progress",
  "status_history": [
    {"status": "triggered", "at": "2026-07-23T14:15:00", "by": "system"},
    {"status": "assigned", "at": "2026-07-23T14:16:00", "by": "system", "to": "周八"}
  ],
}
```

```

    "sub_task_id": "SRC-2026-0089",
    "resolution": null,
    "reflow": {
        "condition": "物料入库且验证通过",
        "notified_to": "李四",
        "notified_at": null,
        "reflow_completed_at": null
    }
}

```

8.7 运行态产出：四张检查结果清单

以上五张孙表是事先设计的**模板数据**（工艺）。当任务实例开始执行后，系统每次跑孙表时逐行生成**检查结果数据**（过程）——后者给执行者提供反馈，也给管理者提供追溯依据。以下为四张清单的数据结构定义。

8.7.1 规则检查问题清单（来源孙表B）

每次触发规则检查时逐条生成。一条规则一次运行产生一条记录。

字段	类型	说明	示例
check_id	string	唯一标识	RULE-CHK-2026-0723-001
timestamp	datetime	检查时间	2026-07-23T14:20:00
step_id	string	所属步骤	STEP2
rule_id	string	规则编号	R4
rule_desc	string	规则描述	生命周期检查
trigger_type	enum	触发方式	field_change / step_enter / manual
target_field	string	目标字段	\$3.3
target_entity	string	目标实体ID	MAT-CON-00521
result	enum	判定结果	pass / warn / block / skip(不触发)
detail	string	具体反馈	XH2.54-16P 即将EOL，最后采购日期2026-12-31
suggested_action	string	建议操作	查找替代 / 评估影响 / 确认保留
resolved	boolean	是否已处理	false
resolved_by	string	处理人	null
resolved_at	datetime	处理时间	null

8.7.2 检查清单问题清单（来源孙表C）

步骤检查时逐条判定生成。一条检查项一次运行产生一条记录。

字段	类型	说明	示例
check_id	string	唯一标识	CHKLST-CHK-2026-0723-005
timestamp	datetime	检查时间	2026-07-23T14:20:00
step_id	string	所属步骤	STEP2
item_id	string	检查项编号	C2
item_desc	string	检查项描述	已选定全部通过约束规则
judge_method	enum	判定方式	auto / manual
result	enum	判定结果	pass / warn / fail / pending
detail	string	汇总反馈	R1✓ R2✓(不触发) R3✓ R4 ⚡ (警告) → 通过(附警告)
affected_rules	[string]	关联规则ID	["R1","R2","R3","R4"]
block_step_flow	boolean	是否阻断步骤流转	false (警告不阻断)

8.7.3 合规检查问题清单（来源孙表D）

人和系统均可触发，以人判断为主。

字段	类型	说明	示例
check_id	string	唯一标识	CMPL-CHK-2026-0723-003
timestamp	datetime	检查时间	2026-07-23T14:20:00
step_id	string	所属步骤	STEP2
item_id	string	合规项编号	L3
item_desc	string	合规要求	新物料须完成DFMEA分析
basis	string	依据文档	产品域设计标准\$5.3
judge_method	enum	判定方式	manual（系统可辅助查询认证状态）
result	enum	判定结果	pass / fail / pending
detail	string	判定说明	E2触发中·待物料入库后人工确认
checked_by	string	判定人	李四
checked_at	datetime	判定时间	null

8.7.4 异常处理记录清单（来源孙表E）

每次异常路径被触发并处理后产生，追踪完整回流过程。

字段	类型	说明	示例
exception_id	string	唯一标识	EXC-2026-0723-002
exception_type	string	异常编号	E2
exception_desc	string	异常描述	物料缺失——电源模块12V→3.3V/5V
trigger_source	enum	触发来源	rule_fail / search_zero / manual
trigger_rule	string	关联规则（规则触发时）	null（检索结果=0触发，非规则触发）
trigger_step	string	触发步骤	STEP2
trigger_field	string	触发字段	§3.2
doc_id	string	关联文档	DOC-0715-005
status	enum	当前状态	triggered / assigned / in_progress / resolved / closed
status_history	[[{status, at, by, to?}]]	状态变更记录	见 §8.6.2 JSON 示例
sub_task_id	string	关联子任务	SRC-2026-0089
resolution	string	处理结果	null
reflow_notified_to	string	回流通知对象	李四
reflow_notified_at	datetime	回流通知时间	null
reflow_completed_at	datetime	回流完成时间	null

以上四张清单是 Part 1 中**运行态**数据的核心组成——它们不是事先设计的模板，而是每次任务执行时系统和人交互产生的不可复用的记录。模板数据（主表/子表/孙表）属于流程+工艺，检查结果数据属于过程。这四张清单也是 Part 1 §1.6 中**演化态闭环**的数据来源——当演化态启动后（§十七），Owner 从这些清单的聚合数据中识别异常模式、驱动工艺修订。

第四部分：设计工位 —— 人在EOS里看到什么、怎样工作

§七~八定义了数据模型。这一部分展示数据在运行态中如何被系统解析、装配和渲染——当李四打开工位时，系统在背后执行的四步流水线（定位→解析→上下文→渲染），以及 TAB 审核菜单如何从孙表自动生成。**四环节视角**：Part 4 聚焦工艺执行环节——李四的工作台。系统运行"工"的部分（资源装配、规则校验、检查判定），支撑人做"艺"的部分（理解需求、选定内容、编写正文）。钱经理的任务策划和监督评价工位在 §十四（端到端回看）和 Part 6（组织落地）中展开。

九、系统在背后做什么 —— 四步流水线

9.1 李四接到一个任务

李四（设计工程师组·硬件设计师）在7月23日早上登录EOS平台。他的任务列表中多了一条："硬件设计方案——XYZ电机项目·电源模块，截止08-05"。

这个任务是怎么到达李四这里的？上一道工序（工序1·用户需求整理）由张三在7月22日完成并发布。系统检测到工序1完成✓→检查主表：工序2（硬件设计方案）的前序依赖=工序1，已满足→工作中心=设计工程师组→通知组内4人→钱经理指定李四负责→任务出现在李四的列表中。

李四点击任务。系统在不到2秒内完成以下四步：

9.2 第一步·定位：找到该读的那一页

系统查询链：

- 任务 ID: TASK-2026-0723-002
 - 查业务启动指令 #BI-2026-0089
 - 构件：电源模块（自研件）
 - 构件文档开发路线：RT-PWR-MODULE-001 v1.0（7月10日创建时锁定）
 - 路线中工序2的定义：
 - 文档类型：DOC-TECH-DESIGN（硬件设计方案）
 - 作业指南：DG-TECH-DESIGN-001 v2.0
 - 工作中心：设计工程师组
 - 前序依赖：工序1（必须）
 - 后续：工序3（参考）+工序5（输入·必须）
 - 查任务执行历史：
 - 任务状态：进行中
 - 当前步骤：Step 1（首次打开，从第一步开始）
 - 本步骤开始时间：无（尚未开始）

定位结果：

作业指南 = DG-TECH-DESIGN-001 v2.0

当前步骤 = Step 1 · 理解需求

文档实例 = DOC-0715-005（硬件设计方案，封面已自动生成，正文为空）

9.3 第二步·解析：翻开作业指南

系统读取作业指南 DG-TECH-DESIGN-001 v2.0 中 Step 1 的五张孙表。对于理解型步骤，孙表的内容相对简洁——没有实体库，没有选用规则，没有异常路径：

Step 1 · 理解需求（理解型步骤）

孙表A·资源清单（5行）：

序号1：输入文档·用户需求整理（角色：输入文档·必须·查场景业务文档序列）

序号2：输入文档·接口定义（角色：输入文档·必须·查场景业务文档序列）

序号3：输入文档·合规要求（角色：合规文档·必须·查场景业务文档序列）

序号4：参考资源·硬件设计规范 v3.1（角色：规范文档·必须·直接引用）

序号5：参考资源·设计方案模板 v2（角色：资源文档·可选·直接引用）

孙表B·规则清单（3行·理解型步骤的通用规则）：

R1.1：输入完整性检查·触发：打开文档时·判定：所有声明"必须"的输入文档状态=已完成

R1.2：版本检查·触发：打开输入文档时·判定：引用的输入文档版本=当前有效版本

R1.3：矛盾标记·触发：保存文档时·判定：正文中标注的矛盾点均附说明

孙表C·检查清单（3行）：

C1.1：所有必须的输入文档已读取确认·自动判定·扫描输入文档的"已读"标记

C1.2：输入文档版本全部当前有效·自动判定·逐份比对版本号

C1.3：矛盾点全部已标记并说明·自动判定·扫描正文临时区

孙表D·合规清单（1行）：

L1.1：遵循硬件设计规范 v3.1 中定义的器件选型原则·人工确认

孙表E·异常清单（2行）：

E1.1：输入文档未就绪·触发：输入文档状态≠已完成·动作：标记 ⏸+正文段落灰化

E1.2：输入文档版本过期·触发：版本≠当前有效·动作：警告 ⚠+加载新版+通知执行者

9.4 第三步·上下文：装配手边的资源

系统遍历孙表A的5行资源声明，在运行时上下文中逐条查询：

声明1："输入文档·用户需求整理·必须·查场景业务文档序列"

查询：本业务启动指令 #BI-2026-0089 的文档序列中

文档类型=DOC-REQ-COMPILE（用户需求整理）

状态=已完成

结果：✓ 找到

文档ID：DOC-0715-001

版本：v2.1

作者：张三

完成日期: 2026-07-22

状态: 已发布

→ 装配到辅助栏·输入文档区:

"用户需求整理 v2.1 ✓ 张三·07-22 [→]"

声明2: "输入文档·接口定义·必须·查场景业务文档序列"

查询: 本场景业务的文档序列中

文档类型=DOC-INTERFACE-DEF (接口定义)

状态=已完成

结果: ✓ 找到

文档ID: DOC-0715-002

版本: v1.0

作者: 张三 (在工序1中作为附件产出)

完成日期: 2026-07-20

状态: 已发布

→ 装配到辅助栏·输入文档区:

"接口定义 v1.0 ✓ 张三·07-20 [→]"

声明3: "输入文档·合规要求·必须·查场景业务文档序列"

查询: 本场景业务的文档序列中

文档类型=DOC-COMPLIANCE-REQ (合规要求)

状态=已完成

结果: ⏳ 未就绪

文档ID: DOC-0715-004

当前状态: 编制中 (负责人: 赵六, 预计07-25完成)

→ 装配到辅助栏·输入文档区:

"合规要求 v3.0 ⏳ 赵六·预计07-25 [→]"

→ 同时注册监听器: 监听DOC-0715-004的状态变更事件

→ 提示: "合规要求尚未完成, §3中引用合规要求的段落暂不可填"

→ 正文§3中标记了"合规引用"的段落自动灰化

声明4: "参考资源·硬件设计规范 v3.1·必须·直接引用"

查询: 资源库中 硬件设计规范 版本 v3.1

结果: ✓ 已定位·当前有效版本

→ 装配到辅助栏·参考资料区:

"硬件设计规范 v3.1 [→]"

声明5: "参考资源·设计方案模板 v2·可选·直接引用"

查询: 模板库中 设计方案模板 版本 v2

结果: ✓ 已定位

→ 装配到辅助栏·参考资料区:

"设计方案模板 v2 [→]"

9.5 第四步·渲染: 工位就绪

硬件设计方案

场景业务进度：

工序1 ✓ — 工序2 ● — 工序3 ○ — 工序4 ○ — 工序5 ○ — 工序6 ○ — 工序7 ○

用户需求 硬件设计 采购技术 安规认证 实现方案 测试报告 验收报告

张三 ✓ 李四 ◀ 王五 赵六 李四 孙七 赵六

本文档步骤：Step1 ● — Step2 ○ — Step3 ○ — Step4 ○

理解需求 选定内容 编写正文 自检与提交

正文工作区

辅助栏

[关闭]

封面表单

单据号 DOC-2026-0715-005 状态 编制中
标题 (待填写) 版本 v0.1
创建人 李四 日期 07-23
部门 设计工程师组

当前步骤

Step 1/4 · 理解需求

操作路径：

逐文档阅读 → 标记矛盾 → 汇总

输入文档 (3)

用户需求整理 v2.1 ✓

张三 · 07-22 [→]

已完成 · 已发布

接口定义 v1.0 ✓

张三 · 07-20 [→]

已完成 · 已发布

合规要求 v3.0 ⌚

赵六 · 预计 07-25 [→]

编制中

⚠ §3 对应段落暂不可填

参考资料 (2)

硬件设计规范 v3.1 [→]

设计方案模板 v2 [→]

完成标准

☐ C1.1 必须输入已确认 (2/3)

☐ C1.2 版本全部有效

☐ C1.3 矛盾点已标记 (0)

合规

☐ L1.1 器件选型原则已确认

输出流转

完成后 → Step 2 选定内容对象

协作

张三(上游) · 在线 [询问]

钱经理(主管) · 在线

正文 TAB

[§1 范围 -] [§2 引用标准 -] [§3 技术要求 -]
[§4 验收条件 -]

§1 范围

[请输入项目背景、适用范围、产品定义 ...]

参考：

· 用户需求整理 v2.1 §1 项目背景

· 用户需求整理 v2.1 §2 功能需求概览

赵六(合规)·离线

【保存草稿】

【预览】

【下一步: Step2 →】

← 灰化 (完成标准未全部满足)

采购管理 > XYZ电机项目 > 电源模块 > 【硬件设计方案】 Step 1

十、工位长什么样——理解型与选用型步骤的页面

系统完成四步流水线后，李四看到的工位页面因步骤类型不同而有不同的布局侧重。以下展示理解型步骤（Step 1）和选用型步骤（Step 2）的完整工位页面，以及 Step 1 的完整操作流。

10.1 Step 1 工位页面——理解型步骤

硬件设计方案

顶部：任务上下文

硬件设计方案 #DOC-0715-005

截止：2026-08-05

场景业务进度

工序1 ✓

工序2 ●

工序3 ○

工序4 ○

工序5 ○

工序6 ○

工序7 ○

用户需求整理

硬件设计方案

采购技术

安规认证

实现方案

测试报告

验收

张三·07-22

李四

王五

赵六

李四

孙七

赵六

← 已完成

← 你在这里

待开始 →

本文档步骤

Step1

Step2

Step3

Step4

理解需求

选定内容对象

编写正文

自检与提交

中左：正文工作区

封面表单

单据号 DOC-2026-0715-005

标题 (待填写)

创建人 李四

部门 设计工程师组

状态 编制中

版本 v0.1

日期 07-23

【编辑】

正文 TAB

【§1 范围 -▼】

【§2 引用标准 -▼】

【§3 技术要求 -▼】

【§4 验收条件 -▼】

§1 范围

辅助栏 [关闭]

当前步骤

Step 1/4 · 理解需求

正常路径：
逐文档阅读→标记矛盾→汇总

当前动作：阅读输入文档

输入文档 (3)

用户需求整理 v2.1 ✓

张三·07-22

已完成·已发布

【请输入项目背景、适用范围、产品定义 ...】

参考：

- 【用户需求整理 v2.1 §1】项目背景
- 【用户需求整理 v2.1 §2】功能需求概览

接口定义 v1.0 ✓
张三·07-20 [→]
已完成·已发布

合规要求 v3.0 ⌚
赵六·预计07-25 [→]
编制中

△ §3中引用合规要求的段落暂

参考资料 (2)

- 硬件设计规范 v3.1
- 设计方案模板 v2

完成标准

- ☐ C1.1 必须输入已确认
合规要求 ⌚ 不计入阻碍
- ☐ C1.2 版本全部有效
用户需求 v2.1 = 当前不
接口定义 v1.0 = 当前不
- ☐ C1.3 矛盾点已标记 (0

合规

- ☐ L1.1 器件选型原则已确

输出流转

- 完成后 → Step 2 选定内容

协作

- 张三(上游)·在线 [询问]
- 钱经理(主管)·在线
- 赵六(合规)·离线

底部：操作栏

- 【保存草稿】
- 【预览】
- 【下一步：Step 2 →】 ← 灰化（完成标准未全部满足）

底部：面包屑

- 采购管理 > XYZ电机项目 > 电源模块 > 【硬件设计方案】 Step 1

53

10.2 Step 2 工位页面——选用型步骤（检索进行中）

当李四进入Step 2后，工位布局切换——辅助栏中出现实体库检索面板，这是选用型步骤特有的：

硬件设计方案

顶部：任务上下文

步骤：Step1 ✓ — Step2 ● — Step3 ○ — Step4 ○

中左：正文工作区

正文 TAB

§1 范围 ✓

§2 引用标准 ✓

§3 技术要求 ⚡ ▼

§4 验收条件 -

§3 技术要求

§3.1 主控芯片

选择物料 ▼

当前：未选定

检索物料库 ...

§3.2 电源模块 ← 光标

选择物料 ▼

要求：12V→3.3V/5V,纹波<50mV,工业级

检索结果：0 条

△ 物料库中未找到满足要求的电源模块

— 下一步操作 —

发起新物料寻源

创建寻源任务，需求参数自动填充

关联到本文档 §3.2

修改检索条件

暂时跳过

§3.3 连接器

型号：XH2.54-16P

料号：MAT-CON-00521

供应商：JST（合格 ✓）

更换查看详情

辅助栏 [关闭]

当前步骤

Step 2/4 · 选定内容对象

检索→比对→验证→写入

当前动作：检索

输入文档 (3)

用户需求整理 v2.1 ✓ 张三

接口定义 v1.0 ✓ 张三

合规要求 v3.0 ⌚ 赵方

实体库检索

搜索物料库 (MAT) ...

当前检索：§3.2 电源模块

条件：12V→3.3V/5V 纹波<50mV 工业级 -40~85°C

△ 未找到满足要求的物料

— 下一步操作 —

发起新物料寻源

修改检索条件

暂时跳过

参考资料 (2)

硬件设计规范 v3.1 [→]

设计方案模板 v2 [→]

完成标准

☐ C1 实体引用全选定 (2/3+1)

54

C3 阻断异常全触发 ✓ C4 人工确认 ? [展开确认]	E2已触发·寻源处理中 (0/2·§3.1 §3.3待确认) 点击→打开确认面板
合规检查 L1 ISO9001 ✓ L2 RoHS ✓ L3 DFMEA ⌚ [查看详情]	← 来自孙表D STMicro+JST认证有效 两个物料均标注RoHS E2触发中·待入库后确认 跳转到合规清单面板
异常处理 E2 物料缺失·处理中 [查看进度]	← 来自孙表E #SRC-2026-0089 创建:07-23 14:15 周八 点击→辅助栏展开异常详情
[运行全部检查]	一键重跑全部孙表

同时，正文 §3.3 段落上有就地标记：

§3.3 连接器

型号：XH2.54-16P（料号 MAT-CON-00521）

供应商：JST（合格 ✓）

⚠ EOL预警：最后采购日期 2026-12-31

当前项目周期预计2026年9月完成，本项目不受影响

[查找替代] [查看详情] [已评估：确认保留]

辅助栏·输入文档区：

输入文档 (3)
用户需求整理 v2.1 ✓ 张三·07-22 [→]
已完成·已发布
接口定义 v1.0 ✓ 张三·07-20 [→]
已完成·已发布
合规要求 v3.0 ⌚ 赵六·预计07-25 [→]
编制中
7月25日 09:00 更新：
⚠ 合规要求已完成 ✓ → 辅助栏状态自动刷新

辅助栏·完成标准区：

完成标准

- ☑ C1 实体引用全选定
 - §3.1 STM32F407VGT6 ✓
 - §3.2 待就绪 ⏳ #SRC-2026-0089
 - §3.3 XH2.54-16P ✓
- ⚠ C2 约束规则全通过（附警告）
 - R1✓ R2跳过 R3✓ R4 ⚡ (§3.3 EOL预警)
 - 警告不阻断流转
- ☑ C3 阻断异常全触发
 - E2 物料缺失→已创建寻源任务 #SRC-2026-0089
 - 处理中·不阻断步骤流转
- ☐ C4 人工确认 (0/2)
 - §3.1 STM32F407VGT6 ☐
 - §3.3 XH2.54-16P ☐
 - 请与项目主管确认选型后勾选

10.4 Step 1 的完整操作流程示例

李四在Step 1中做一个理解型步骤的典型操作：

- ① 李四看到辅助栏输入文档区有两份已就绪的文档（用户需求整理、接口定义）
一份待就绪（合规要求 ⏳）
- ② 点击"用户需求整理 v2.1 [→]"
→ 辅助栏展开文档预览：
关键章节摘要：
 - §1 项目背景：XYZ电机项目需要一款新型电源模块 ...
 - §2 功能需求：输入12V，输出3.3V/5V双路，纹波<50mV ...
 - §3 性能需求：效率>85%，工业级温度范围-40~85°C ...
 【打开完整文档】
- ③ 在正文 §1 中开始填写项目背景
系统在正文中自动生成内联引用标记：
"本项目为XYZ电机项目配套电源模块开发，具体需求参见
【用户需求整理 v2.1 §1 项目背景】"
- ④ 点击TAB [§1 范围 -▼]（首次打开，状态为"-"表示未运行过检查）
→ 下拉菜单只有两项（理解型步骤）：
 - 完成检查
 - C1.1 必须输入已确认 (2/3) ← 合规要求未就绪不计入
 - C1.2 版本全部有效 ✓
 - C1.3 矛盾点已标记 (0) ← 尚未发现矛盾点
 - 合规检查
 - L1.1 器件选型原则已确认 ?
 【运行全部检查】
- ⑤ 点击 【运行全部检查】

系统跑 Step1 的孙表B（3条规则）和孙表C（3条检查项）：

规则R1.1·输入完整性检查：
用户需求整理 v2.1 → 状态=已完成 ✓
接口定义 v1.0 → 状态=已完成 ✓
合规要求 v3.0 → 状态=编制中 X → 但不是阻断，C1.1只计"必须"输入
→ 结果：2/3必须输入已就绪（合规要求未就绪不阻断步骤1）

规则R1.2·版本检查：
用户需求整理 v2.1 → 当前有效版本=v2.1 ✓
接口定义 v1.0 → 当前有效版本=v1.0 ✓
→ 全部通过 ✓

规则R1.3·矛盾标记：
扫描正文临时区 → 无矛盾标记 → 通过（空集通过）

检查C1.1 → 2/3 已确认（合规要求不计入阻断） → 通过 ✓
检查C1.2 → 版本全部有效 → 通过 ✓
检查C1.3 → 无矛盾项 → 通过 ✓

→ 菜单刷新：全部✓
→ TAB标签变为 [§1 范围 ✓]
→ 辅助栏完成标准区全部打勾
→ [下一步：Step2 →] 亮起

⑥ 李四依次完成 §2/§3/§4 的初始填写
（§3 中引用合规要求的部分保持灰化）
每完成一个TAB → [运行检查] → 通过 → TAB变✓

⑦ 全部TAB ✓ → 顶部步骤条 Step 1 ✓
→ 李四点击 [下一步：Step2 →]
→ 系统锁定Step1内容（只读）
→ 加载Step2的五张孙表 → 刷新工位
→ Step2 开始（选用型步骤，辅助栏展开实体库检索面板）

十一、TAB菜单 —— 孙表数据如何变成操作界面

11.1 菜单的每一项来自哪张孙表

以 Step 2（选用型步骤）为例，展示系统如何从五张孙表生成菜单。菜单不是设计者手工配置的——是孙表数据的UI投影。

孙表B（规则清单）→ 一级菜单"规则检查"：

孙表B数据行	菜单中的子项
编号:R1 描述:优先复用	└ R1 优先复用

触发:候选物料满足全部参数要求
判定:直接选用(推荐不强制)
阻断级别:推荐
不通过→异常:-

状态标记:需要查询上次运行结果缓存
点击行为:单独运行R1
运行范围:对当前步骤所有已选定物料逐条判定

编号:R2 描述:三家比价
触发:成本占比>5%或单价>¥500
判定:候选供应商≥3家
阻断级别:阻断
不通过→异常:E1

└ R2 三家比价
状态标记:同上

编号:R3 描述:供应商在册
触发:选定物料后
判定:物料.供应商∈AVL
阻断级别:阻断
不通过→异常:E3

└ R3 供应商在册
状态标记:同上

编号:R4 描述:生命周期检查
触发:选定物料后
判定:已停供→阻断/即将EOL→警告/
量产→通过
阻断级别:情况而定
不通过→异常:E4

└ R4 生命周期检查
状态标记:同上

孙表C (检查清单) → 一级菜单"完成检查":

孙表C数据行

菜单中的子项

编号:C1 描述:所有实体引用字段均已选定
判定:扫描正文字段类型="实体引用"
判定方式:自动

└ C1 实体引用全选定
状态标记:上次运行时 (2/3)
点击行为:单独运行C1
运行范围:扫描正文§3.1/§3.2/§3.3

编号:C2 描述:已选定全部通过约束规则
判定:逐条重跑R1~R4
判定方式:自动

└ C2 约束规则全通过
状态标记:汇总规则检查结果
点击行为:重跑全部规则+汇总

编号:C3 描述:阻断型异常全部已触发
判定:所有阻断型异常的action已执行
判定方式:自动

└ C3 阻断异常全触发
状态标记:检查异常处理记录清单
点击行为:重新检查异常触发状态

编号:C4 描述:关键器件选型已与项目主管确认
判定:检查"关键器件"标记+确认状态
判定方式:人工确认

└ C4 人工确认
状态标记:(0/2)待确认
点击行为:展开确认面板

孙表D (合规清单) → 一级菜单"合规检查":

编号:L1 供应商须通过ISO9001

└ L1 ISO9001认证

编号:L2 电源模块须满足RoHS 2.0

└ L2 RoHS 2.0

孙表E (异常清单) → 一级菜单"异常处理":

编号:E2 物料缺失·处理中

L E2 物料缺失·处理中

触发:检索结果=0

点击行为:跳转到辅助栏异常处理入口

状态:寻源任务#SRC-2026-0089进行中

查看完整的处理记录和回流状态

11.2 一次完整检查的详细过程**T0 7月23日 11:00 · Step2 初始状态**

李四刚从Step1进入Step2。正文§3.1/§3.2/§3.3全部标记"待选定"。

TAB [§3 技术要求 -] (未运行过检查)。

点击TAB → 下拉菜单:

所有条目都是 "-" (未运行)

T1 7月23日 11:30 · 李四选定§3.1物料

光标在§3.1 → 辅助栏·实体库检索面板激活

→ 检索物料库(MAT): ARM Cortex-M4 168MHz

→ 返回3个候选: STM32F407VGT6(¥52) / ATSAM4E8EA-AU(¥48) / LPC1788FBD208(¥55)

李四点击[并排比对] → 选定 STM32F407VGT6

→ 正文§3.1更新:

主控芯片: STM32F407VGT6 (料号 MAT-IC-00342)

Cortex-M4, 168MHz, 1MB Flash

[更换] [查看详情]

T2 7月23日 14:00 · 李四选定§3.3物料

光标在§3.3 → 检索物料库: 2.54mm间距 16Pin 带锁扣

→ 选定 XH2.54-16P (料号 MAT-CON-00521, 供应商JST)

→ 正文§3.3更新

T3 7月23日 14:15 · 李四在§3.2检索—触发异常

光标在§3.2 → 检索物料库: 12V→3.3V/5V 纹波<50mV 工业级

→ 0条结果

辅助栏·实体库检索面板底部出现:

— 未找到满足要求的电源模块 —

[发起新物料寻源] [修改检索条件] [暂时跳过]

李四点击[发起新物料寻源]

→ 系统按孙表E的E2声明自动创建寻源任务

→ 寻源任务 #SRC-2026-0089 已创建

→ 正文§3.2自动更新:

电源模块: [待就绪 ⏳ #SRC-2026-0089]

要求: 12V→3.3V/5V, 纹波<50mV, 工业级 -40~85°C

→ 辅助栏·检查清单区更新

T4 7月23日 14:20 · 李四点击TAB → [运行全部检查]

系统开始跑 Step2 的全部孙表。以下是完整执行记录:

规则检查

R1·优先复用

对已选定物料逐条判定:

§3.1 STM32F407VGT6 → 物料库中存在 ✓

§3.2 待就绪 → 跳过

§3.3 XH2.54-16P → 物料库中存在 ✓

→ 结果: 2通过 1跳过 → ✓

R2·三家比价

对已选定物料逐条判定触发条件:

§3.1 STM32F407VGT6 → 单价¥52 < ¥500 → 不触发

§3.3 XH2.54-16P → 单价¥8 < ¥500 → 不触发

→ 结果: 无触发 → ✓

R3·供应商在册

对已选定物料查询AVL:

§3.1 STM32F407VGT6 → ST Micro → 查AVL → 在册 ✓

§3.3 XH2.54-16P → JST → 查AVL → 在册 ✓

→ 结果: 2通过 ✓

R4·生命周期检查

对已选定物料查询生命周期状态:

§3.1 STM32F407VGT6 → 量产 → 通过 ✓

§3.3 XH2.54-16P → 即将EOL → 警告 ⚡

最后采购日期: 2026-12-31

→ 结果: 1通过 1警告 → 警告 ⚡

→ E4不触发阻断 (即将EOL不阻断)

规则检查总结: 4/4规则已跑, 3通过 1警告

完成检查

C1·所有实体引用字段均已选定

扫描正文中字段类型="实体引用"的字段：

§3.1 主控芯片 → STM32F407VGT6 → 非空 ✓

§3.2 电源模块 → [待就绪 ⏳] → 非空但引用目标暂不存在
判定规则：待就绪状态不阻断C1 → 按已选定处理 ✓

§3.3 连接器 → XH2.54-16P → 非空 ✓

→ 结果：(3/3 + 1待就绪) → ✓

C2·已选定全部通过约束规则

汇总规则检查结果：

R1✓ R2✓(不触发) R3✓ R4 ⚡ (警告)

阻断型规则 (R2/R3/R4阻断) → 无阻断型不通过

警告 (R4 EOL预警) → 不阻断C2

→ 结果：通过 (附警告) ⚠

C3·阻断型异常全部已触发

检查异常处理记录清单：

E2 (物料缺失) → 已触发 → 寻源任务#SRC-2026-0089处理中

E1/E3/E4 → 未触发

所有已触发的阻断型异常的action已执行✓

→ 结果：通过 ✓

C4·关键器件选型已与项目主管确认

检查"关键器件"标记的物料：

§3.1 STM32F407VGT6 → 标记为关键器件 → 未确认

§3.3 XH2.54-16P → 标记为关键器件 → 未确认

→ 结果：(0/2) → 待确认 ?

完成检查总结结果：C1✓ C2⚠ C3✓ C4?

合规检查

L1·ISO9001认证

ST Micro → 查供应商认证库 → ISO9001:2015 有效期至2027

JST → 查供应商认证库 → ISO9001:2015 有效期至2026

→ 结果：全部合规 ✓

L2·RoHS 2.0

STM32F407VGT6 → 查物料产品页 → RoHS标记 ✓

XH2.54-16P → 查物料产品页 → RoHS标记 ✓

→ 结果：全部合规 ✓

L3·DFMEA分析

§3.2 已触发E2新物料寻源

→ 结果：触发中 ⏳ (需在物料入库并验证后人工确认)

合规检查总结结果：L1✓ L2✓ L3 ⏳

异常处理

E2·物料缺失

寻源任务 #SRC-2026-0089

创建时间：2026-07-23 14:15

描述：电源模块 12V→3.3V/5V 纹波<50mV 工业级-40~85°C

关联段落：§3.2

当前状态：处理中（已分派给寻源工程师·周八）

→ 结果：处理中 ⌚

异常处理总结结果：E2 ⌚

T5 14:20 · 菜单刷新

[§3 技术要求 ⚡ ▼] ← TAB标签变为 ⚡ （C2有警告 + C4待确认）

规则检查

- └ R1 优先复用 ✓
- └ R2 三家比价 ✓
- └ R3 供应商在册 ✓
- └ R4 生命周期 ⚡ ← §3.3 EOL预警

完成检查

- └ C1 实体引用全选定 ✓ ← (3/3 + 1待就绪)
- └ C2 约束规则全通过 ⚠ ← R4有警告
- └ C3 阻断异常全触发 ✓ ← E2已触发
- └ C4 人工确认 ? ← 0/2

合规检查

- └ L1 ISO9001 ✓
- └ L2 RoHS ✓
- └ L3 DFMEA ⌚

异常处理

- └ E2 物料缺失·处理中 ⌚

[运行全部检查]

T6 14:21 · 李四处理未通过项

点击 ⚡ R4 生命周期 → 展开详情：

§3.3 连接器 XH2.54-16P（料号 MAT-CON-00521）

生命周期状态：即将EOL

最后采购日期：2026-12-31

建议：确认替代方案，或点击[查找替代]

此警告不阻断步骤流转

李四点击[查找替代] → 系统检索兼容替代物料 → 找到2个替代候选

李四评估后决定暂时保留XH2.54-16P（库存充足，项目周期内不受影响）

→ 标记："已评估替代方案，当前项目周期内不受EOL影响"

点击 ? C4 人工确认 → 展开确认面板：

§3.1 STM32F407VGT6（关键器件）→ [联系主管]

§3.3 XH2.54-16P（关键器件）→ [联系主管]

李四通过协作面板联系钱经理 → 钱经理确认选型 ✓

李四在确认面板上逐条勾选 → C4状态变为 ✓

T7 14:30 · 再次 [运行全部检查]

全部孙表重新跑一遍：

规则检查 → R4 仍然 ⚡（EOL警告持续存在，但李四已标记说明）

完成检查 → C1✓ C2⚠ C3✓ C4✓

TAB标签：从 ⚡ 变为 ⚡（C2的警告仍在，但非阻断）

[下一步：Step3 →] 亮起 ← C4通过后满足条件

李四点击 [下一步] → Step2完成 ✓ → Step3 开始

11.3 文档进度是怎么算出来的

以T6时刻的状态计算文档进度：

Step1·理解需求（权重15%）：

C1.1✓ C1.2✓ C1.3✓ → $3/3 = 100\%$

→ 贡献： $15\% \times 100\% = 15\%$

Step2·选定内容对象（权重35%·选用型步骤权重重大）：

C1 实体引用全部选定 → ✓ → 100%

C2 约束规则全部通过 → ⚠（警告不扣分） → 100%

C3 阻断异常全部触发 → ✓ → 100%

C4 人工确认 → ✓ → 100%

Step2完成度 = 100%

→ 贡献： $35\% \times 100\% = 35\%$

Step3·编写正文（权重35%）：

正文已开始填但未运行过检查 → 完成度 = 0%

→ 贡献： $35\% \times 0\% = 0\%$

Step4·自检与提交（权重15%）：

未开始 → 完成度 = 0%

→ 贡献： $15\% \times 0\% = 0\%$

文档总进度： $15\% + 35\% + 0\% + 0\% = 50\%$

这个50%是系统自动计算出来的——不需要李四填报，不需要钱经理追问。管理者在场景业务视图中看到的就是这个数字。

十二、辅助栏 —— 七种资源点开之后看到什么

孙表A的每条资源声明了资源类别。系统不仅按类别决定"怎么获取"，也按类别决定"点击后展示什么"。执行者在辅助栏中点击任意资源，展开的内容取决于资源类别。

12.1 输入文档下钻

辅助栏·输入文档区：

- ▶ 输入文档 (3)
 - 用户需求整理 v2.1 ✓ 张三·07-22 [→]
 - 接口定义 v1.0 ✓ 张三·07-20 [→]
 - 合规要求 v3.0 ⌚ 赵六·预计07-25 [→]

点击"用户需求整理 v2.1 [→]" → 辅助栏展开文档预览：

用户需求整理 v2.1 · 预览

作者：张三 完成日期：2026-07-22 状态：已发布

§1 项目背景

XYZ电机项目需要一款新型电源模块，用于下一代电机驱动系统。
当前使用的电源模块存在效率低（78%）、纹波大（120mV）等问题。

§2 功能需求

- 输入电压：12V DC

- 输出电压：3.3V/1A + 5V/2A 双路输出

- 纹波要求：<50mV

- 效率要求：>85%

§3 性能需求

- 工作温度范围：-40°C ~ +85°C（工业级）

- 电磁兼容：符合IEC 61000-4标准

[打开完整文档]

[收起 ▲]

12.2 实体库检索面板激活

辅助栏·实体库（孙表A声明了3个实体库：MAT/REUSE/AVL）：

- ▶ 实体库检索

[搜索物料库(MAT) ...]

当前检索：§3.1 主控芯片

检索条件：ARM Cortex-M4 168MHz 1MB Flash

65

检索结果 (3):

- STM32F407VGT6 Cortex-M4 168MHz 1MB ¥52 TI
- ATSAM4E8EA-AU Cortex-M4 120MHz 2MB ¥48 Microchip
- LPC1788FBD208 Cortex-M3 120MHz 512KB ¥55 NXP ⚠
(LPC1788不满足Cortex-M4要求, 已自动标注)

[并排比对A vs B] [选定A] [选定B]

— 以上候选均不满足需求? —

[发起新物料寻源] ← 一直可见 (孙表E E2的入口)

12.3 工作中心下钻

辅助栏·工作中心区 (如果孙表A声明了工作中心资源):

► 工作中心

设计工程师组 (4人)

[展开 ▼]

点击展开:

工作中心·设计工程师组

当前任务负载: 3/4 人在忙, 1 人空闲

本周完成: 2份文档 | 进行中: 3份文档

人员	当前任务	状态	可用
李四	硬件设计方案·Step2 XYZ电机项目·电源模块	进行中 (当前)	-
王五	实现方案·§1 范围 ABC控制器项目	进行中	-
赵六	测试报告·Step3 XYZ电机项目·电机驱动器	进行中	-
孙七	-	空闲	✓

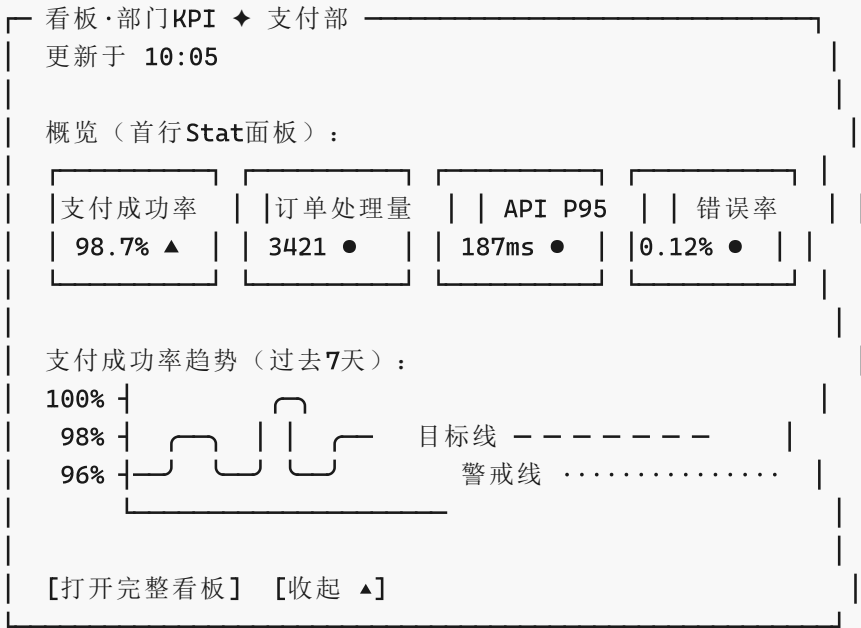
[收起 ▲]

12.4 看板下钻

辅助栏·参考资料区（如果孙表A声明了看板资源）：

- ▶ 参考资料
- 部门KPI看板 [→]
- 硬件设计规范 v3.1 [→]
- 设计方案模板 v2 [→]

点击"部门KPI看板 [→]" → 辅助栏展开看板图示：



12.5 指标下钻

辅助栏·参考资料区：

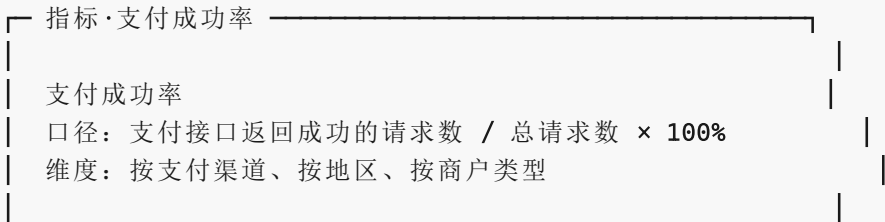
- ▶ 参考资料
- 公司级指标清单 v1.0 [→]

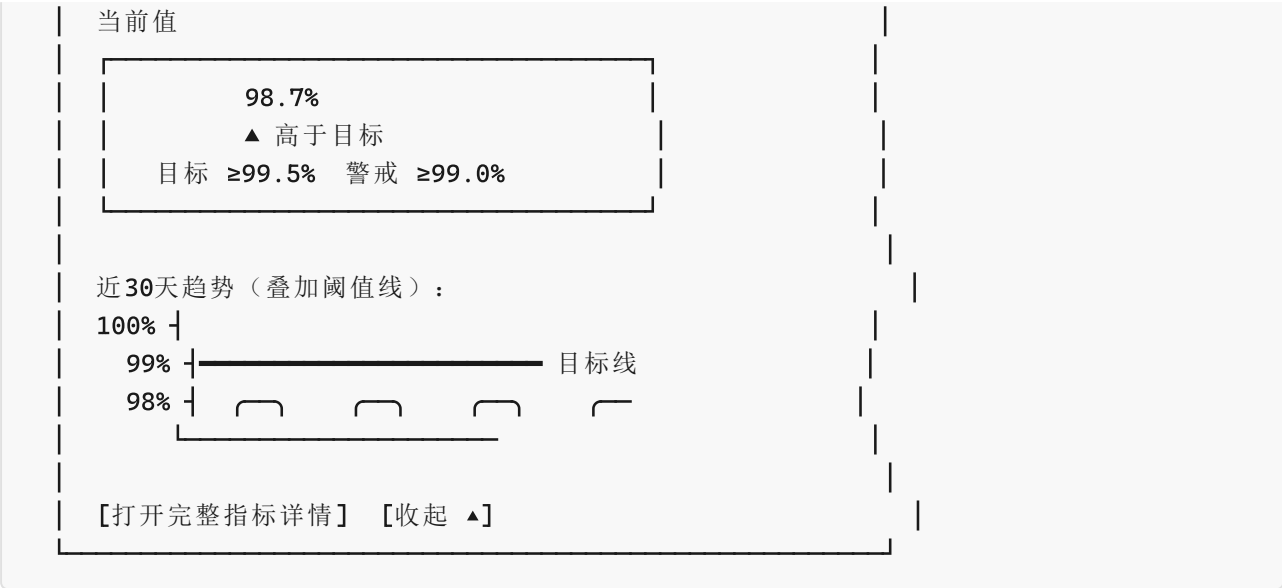
点击 → 展开指标列表：

指标定义库(IND) 中匹配"支付部"的指标 (4):

支付成功率	当前98.7%	目标≥99.5%	[→]
订单处理量	当前3421/min	目标>3000	[→]
API P95延迟	当前187ms	目标<200ms	[→]
错误率	当前0.12%	目标<0.1%	[→]

点击"支付成功率 [→]" → 辅助栏展开指标详情：





12.6 模板下钻

辅助栏·参考资料区：

▶ 参考资料

设计方案模板 v2

[→]

点击展开：

设计方案模板 v2 · 结构预览

封面表单字段：

单据号（自动生成） / 标题（必填） / 状态（自动）

版本（自动） / 创建人（自动） / 日期（自动）

正文结构：

§1 范围 — 富文本，必填

§2 引用标准 — 表格，可选

§3 技术要求 — 含实体引用字段，必填

§3.1 主控芯片 — 实体引用·物料库(MAT)

§3.2 电源模块 — 实体引用·物料库(MAT)

§3.3 通信接口 — 实体引用·复用构件库(REUSE)

§3.4 性能参数 — 表格，必填

§4 验收条件 — 富文本，必填

[应用此模板创建新文档]

[收起 ▲]

12.7 资源清单的三层下钻总表

资源类别	一级显示	二级展开	三级跳转
输入文档	清单项·名称+版本+作者	文档预览·关键章节摘要	打开完整文档
实体库	检索入口·搜索框	检索结果+候选列表	打开实体详情页

68

工作中心	名称+人数	+比对面板+约束校验 人员状态列表	—
看板	名称+受众+更新频率	每人任务+可用状态 看板图示摘要	打开完整看板
指标	名称+口径摘要+当前值	首行 Stat +关键趋势 指标图示	打开完整指标详情
模板	名称+版本	当前值+趋势+阈值线 模板正文结构预览	应用模板到正文
参考文档	名称+版本+摘要	关键字段列表 文档内容预览	新建文档实例
		关键章节	打开完整文档 新窗口/新 TAB

第五部分：端到端演示 —— 跟着一个构件在EOS中走完全程

以上各部分分别定义了映射关系、数据结构、交互机制。这一部分用一个完整的端到端场景——"电源模块开发，从编制路线到整条线完成"——**将四个环节串联演示**：赵总工编制路线（工艺开发）→ 钱经理启动项目和监控进度（任务策划+监督评价）→ 张三/李四/王五执行开发（工艺执行），完整走通 §1.5 的四环节闭环（§十三）。然后从工位对比、管理者视图和角色感知三个视角对场景进行总结分析（§十四）。

十三、全程记录 —— 电源模块从路线到交付

13.1 角色表

角色	姓名	身份	在本场景中的职责
构件设计负责人	赵总工	工艺工程师	编制构件文档开发路线
项目主管	钱经理	生产计划员	启动项目、分配任务、监控进度
系统工程师	张三	工序1操作者	开发"用户需求整理"
硬件设计师	李四	工序2操作者	开发"硬件设计方案"
采购工程师	王五	工序3操作者	开发"采购技术规范"
质量工程师	赵六	工序4操作者	开发"安规认证报告"
寻源工程师	周八	异常路径-E2接收者	处理李四发起的新物料寻源
技术负责人	—	评审节点	各文档的技术评审

13.2 第一幕：赵总工编制路线（7月1日）

上午 09:00。赵总工登录EOS平台，打开"构件工艺路线设计器"。

他的起点是"为已有构件设计路线"——电源模块（自研件）之前没有正式编制过构件文档开发路线，之前是按项目临时组织的文档序列。现在需要把它固化为可复用的工艺路线。

第一步：选定构件。

赵总工选择构件"电源模块（自研件）"→系统自动加载：

- PL2 项目类型：设备产品（决定了路线模式：需求→设计→采购→实现→测试→验收）

- PL3 构件类型：自研件（决定了选用型步骤的实体库集合和约束规则集）
- PL4 WBS：电源模块
- 已有的前序文档定义：无（首次编制）

第二步：系统推导候选路线。

系统基于四层信息自动推导：

推导源	推导结果
自研件路线模板 v1.0 → 6道工序的骨架	工序框架：需求类/设计类/采购类/实现类/测试类/验收类
设备产品类型偏好	工序3（采购类）必须·涉及新物料采购 工序2（设计类）使用"设计方案"而非"概念设计"
同类构件路线： "电机驱动器(自研件) v1.5" "电源模块(自研件)·旧版"	工序1文档类型 = "用户需求整理"（来自电机驱动器路线） 工序2文档类型 = "设计方案"（来自电源模块旧有实践） 工序3文档类型 = "采购技术规范" 工序5文档类型 = "测试报告" 工序6文档类型 = "验收报告"
文档类型定义（26号资产）： 支持信息引用声明 → 输入检查 正文字段定义 → 模板和字段约束 处理配置 → 流转路径	每道工序关联的作业指南自动推导

推导完成度：85%。6道工序的骨架+文档类型+工作中心已自动填充。需人工确认/补全：

- 工序4的文档类型：系统推荐"实现方案"（匹配度85%），赵总工确认
- 工序5的文档类型：系统推荐"测试报告"（匹配度90%），赵总工确认
- 是否额外需要合规类文档？赵总工判断需要——法规要求电源模块必须具备安规认证

第三步：调整。

赵总工在工序3和工序4之间插入新工序"安规认证报告"。系统根据上下文（采购技术规范→实现方案之间）和自研件路线模板推荐候选文档类型→赵总工选定"安规认证报告（DOC-CERT-REPORT）"→系统自动关联作业指南 DG-CERT-REPORT-001 v1.0→后续工序自动后移。

赵总工审阅每道工序的作业指南详情：

- 工序1（用户需求整理·理解型+编写型）：全部自动推导，✓ 确认
- 工序2（设计方案·选用型）：Step2的实体库声明✓、约束规则R1~R4✓、§2填写指引需补充 → 赵总工补充
- 工序3（采购技术规范·选用型）：Step2约束规则R1~R4✓、§3编写指引需补充 → 赵总工补充
- 工序4~7：全部自动推导 ✓ 确认

第四步：发布。

赵总工点击[发布路线 v1.0]。系统检查：7道工序文档类型✓、工作中心✓、前序依赖无循环✓、作业指南全部确认✓→发布。

系统自动：路线状态草稿→已发布、生效日期2026-07-01、路线版本v1.0。赵总工关闭设计器，做别的事去了。

13.3 第二幕：钱经理启动项目（7月10日）

上午 10:00。XYZ电机项目进入电源模块开发阶段。钱经理登录EOS，打开"业务启动"。

钱经理选定场景业务模板"电源模块开发"→系统加载构件文档开发路线 v1.0（7月1日赵总工发布的版本）→展示7道工序的文档序列和前后依赖。

钱经理逐工序指定：

- 工序1·用户需求整理：张三（需求分析组）·07-22
- 工序2·硬件设计方案：李四（设计工程师组）·08-05
- 工序3·采购技术规范：王五（采购工程师）·08-15
- 工序4·安规认证报告：赵六（质量工程师）·08-10
- 工序5·实现方案：李四（设计工程师组）·08-25
- 工序6·测试报告：孙七（测试工程师组）·09-05
- 工序7·验收报告：赵六（质量工程师）·09-10

优先级：高（XYZ电机项目的关键路径）。约束：预计工序2会触发新物料寻源。

钱经理点击[启动]。系统执行：

- 创建业务启动指令 #BI-2026-0089
- 锁定路线 v1.0（后续路线升级不影响这批活）
- 逐工序创建任务实例
- 工序1无前序依赖→状态="可开始"→派发给张三
- 工序2~7前序不满足→状态="等待前序"
- 通知张三："你有新任务：用户需求整理·XYZ电机项目·电源模块·07-22截止"

13.4 第三幕：张三开发第一份文档（7月11日→22日）

7月11日 09:00。张三登录→任务列表出现新任务→点击→工位打开。

系统定位：工序1·用户需求整理→子表 DG-REQ-COMPIL-001 v1.2→Step 1。

孙表A资源解析：输入文档（无——首份文档无前序），参考资源（通用设计规范 v1.0 + 需求整理模板 v2）→装配到辅助栏。

张三在Step1中：阅读参考资源→在正文\$1/\$2/\$3中填写需求。TAB上[运行检查]→全部通过→[下一步]亮起。

7月11日~14日：张三依次完成Step1→Step2→Step3。每步以检查结束。

7月14日 16:00：张三完成全部步骤→[提交技术评审]。系统触发处理配置：编制→技术评审（李工）→批准（钱经理）。

7月15日 10:00：李工完成技术评审→通过（2条修改意见）。张三修改→重新提交。

7月16日 09:00：钱经理批准→文档发布 v1.0。系统自动：

- 工序1完成✓
- 检查主表：工序2前序依赖=工序1→已满足→工序2解锁
- 通知设计工程师组4人→钱经理分派给李四

13.5 第四幕：李四开发第二份文档（7月23日→30日）

7月23日 09:00。李四登录→任务："硬件设计方案·XYZ电机项目·电源模块·08-05截止"→打开工位。

系统定位：工序2→子表 DG-TECH-DESIGN-001 v2.0→Step 1（理解型步骤）。

Step1•理解需求（7月23日）：

孙表A装配：输入文档=张三的用户需求整理 v2.1 ✓ + 接口定义 v1.0 ✓ + 合规要求 v3.0 ⌚ →辅助栏输入文档区显示。参考资源=硬件设计规范 v3.1 →辅助栏参考资料区显示。

李四点击"用户需求整理 [→]"→辅助栏展开预览→阅读需求→在正文中标记矛盾点和注意事项。Click TAB [运行检查]→C1.1✓ C1.2✓ C1.3✓→Step1完成。

Step2•选定内容对象（7月23日→24日）：

孙表A装配→辅助栏实体库检索面板激活（物料库MAT + 复用构件库REUSE + AVL）。

7月23日 11:00·李四光标在§3.1主控芯片→辅助栏检索：ARM Cortex-M4 168MHz→3个候选→选定STM32F407VGT6（料号 MAT-IC-00342, ST Micro, ¥52）。正文§3.1自动更新。

7月23日 14:00·李四光标在§3.3连接器→检索→选定XH2.54-16P（料号 MAT-CON-00521, JST, ¥8）。正文§3.3自动更新。

7月23日 14:15·李四光标在§3.2电源模块→检索：12V→3.3V/5V 纹波<50mV 工业级→0条结果。

辅助栏底部出现异常路径入口：

— 未找到满足要求的电源模块 —
[发起新物料寻源] [修改检索条件] [暂时跳过]

李四点击[发起新物料寻源]。系统按孙表E E2声明执行：

- 创建寻源任务 #SRC-2026-0089
- 任务描述自动从§3.2提取："电源模块 12V→3.3V/5V 纹波<50mV 工业级"
- 关联锚点：§3.2 ↔ #SRC-2026-0089

- 正文§3.2更新: "电源模块: [待就绪 ⏰ #SRC-2026-0089]"
- 不阻断步骤流转

7月23日 14:20·李四点击TAB→[运行全部检查]。系统跑孙表B/C/D/E。

规则检查结果:

- R1优先复用: STM32F407✓ XH2.54-16P✓ → ✓
- R2三家比价: 两个物料单价均<¥500→不触发→✓
- R3供应商在册: ST Micro∈AVL✓ JST∈AVL✓→✓
- R4生命周期检查: STM32F407量产✓ XH2.54-16P即将EOL⚠

完成检查结果:

- C1实体引用全选定: (3/3+1待就绪)→✓
- C2约束规则全通过: R4有警告→通过(附警告)⚠
- C3阻断异常全触发: E2已触发✓→✓
- C4人工确认: (0/2)→?

合规检查结果: L1 ISO9001✓ L2 RoHS✓ L3 DFMEA ⏰ (E2触发中)

菜单刷新。TAB标签: [§3 技术要求 ⚡]。

7月23日 14:25·李四处理R4警告→查找替代→评估后保留XH2.54-16P→标记说明。

7月23日 14:30·李四通过协作面板联系钱经理→确认两个关键器件选型→在C4面板上逐条勾选。

7月23日 14:35·再次[运行全部检查]→C4✓→全部通过 (C2附警告不阻断) →[下一步: Step3→]亮起。

Step3·编写正文 (7月24日): 李四按作业指南§1~§4模板填写正文。TAB上逐节[运行检查]→全部通过。

Step4·自检与提交 (7月24日 16:00): [运行全部检查]→全部通过→[提交流转: 技术评审]。

7月25日 10:00: 技术评审通过 (1条修改意见: §3.3 EOL预警需补充替代方案评估)。李四补充替代方案评估说明→重新提交。

7月25日 16:00: 批准通过→文档发布 v1.0。系统自动:

- 工序2完成✓
- 检查主表: 工序3前序依赖=工序1+工序2→全部满足→工序3解锁
- 工序4前序依赖=工序3→工序3未完成→继续等待
- 通知采购工程师组→钱经理分派给王五

7月30日 09:00: 寻源任务 #SRC-2026-0089 完成。周八找到满足要求的物料 LMR14030 (纹波30mV, 12V→3.3V/5V) →入库→物料号 MAT-POW-00128。

系统检测到回流条件满足→通知李四："§3.2 电源模块的寻源已完成，候选物料已入库"。

李四打开硬件设计方案→工位加载→§3.2状态从"待就绪 ⏳"变为"候选物料就绪"→在辅助栏检索面板中出现 LMR14030→选定→校验R1~R4全部通过→正文§3.2更新→保存→硬件设计方案 v1.1 发布（变更类型：增量型·仅§3.2更新）。

系统检测到硬件设计方案版本更新→通知王五："输入文档'硬件设计方案'已更新到v1.1"。

13.6 第五幕：后续工序推进（7月29日→9月）

第四幕详细展示了工序2（设计方案·选用型）的完整执行过程。后续5道工序各有不同的文档类型、步骤结构和工作中心——以下聚焦每道工序的**差异点**，而非重复相同的"打开工位→辅助栏装配→做→查→过"模式。

工序3·采购技术规范（王五·采购工程师组·7月29日→8月15日）

与工序2同样为选用型步骤，但有三处关键差异：

差异1——实体库不同：工序2检索的是物料库（MAT）和复用构件库（REUSE），工序3主要检索****合格供应商名录（AVL）****和物料库（MAT）。王五的 Step2 检索面板默认优先检索 AVL——因为采购技术规范的核心任务是"为选定的物料匹配合格供应商"，而非从零选型。

差异2——规则集不同：工序3的孙表B规则侧重于供应商维度：

- R1·供应商在册（所有选定供应商 $\in$ AVL — 阻断）
- R2·供应商绩效（最近12个月交货准时率 $\geq 95\%$ ，质量合格率 $\geq 98\%$ — 警告）
- R3·供应商产能（当前负载 $\leq 80\%$ — 警告）
- R4·地域合规（涉及出口管制的物料，供应商所在国需在合规白名单 — 阻断）

这与工序2的 R1R4（优先复用/三家比价/供应商在册/生命周期检查）不同——虽然都叫 R1R4，但规则内容由文档类型的作业指南决定。

差异3——输入文档的多版本处理：王五在7月29日打开工位时，辅助栏显示硬件设计方案 v1.0。7月30日李四发布 v1.1→系统通知王五→辅助栏自动刷新到 v1.1。王五需要关注的变更：§3.2 电源模块从"待就绪"变为"已选定 LMR14030"→这直接改变了他的任务——v1.0 中 §3.2 没有选定物料，他不需要为电源模块匹配供应商；v1.1 中有了物料，他需要为 LMR14030 的新供应商做准入检索。

8月15日：王五完成采购技术规范→技术评审通过→文档发布。工序3完成✓。

工序4·安规认证报告（赵六·质量工程师组·8月01日→8月10日）

工序4是一个**合规型步骤为主**的文档——与工序2/3的选用型步骤有本质区别：

步骤类型差异：安规认证报告的作业指南中，Step2 不是"选定内容对象"（没有实体库检索），而是"逐条核对法规要求"——一个以孙表D（合规清单）为主导的步骤。赵六的工作流是：打开工位→辅助栏展示本产品适用的法规清单（孙表D·合规清单，每项标记适用/不适用/待确认）→逐条查阅法规原文→在正文中记录合规判定和证据→TAB上逐项标记合规状态。

没有实体库检索，但有法规库检索：辅助栏中没有物料库/AVL检索面板，取而代之的是法规库（REG）——赵六可以按产品类别+目标市场检索适用的安规标准。这展示了辅助栏的资源类别随文档类型动态切换——不是所有文档都有7种资源，而是作业指南的孙表A声明了什么就展示什么。

并行启动的机理：工序4的前序依赖是工序3（采购技术规范），但赵六在8月01日就能开始工作——因为安规认证的关键输入是"产品设计参数"（来自工序2·已完成）和"目标市场"（来自工序1·已完成），工序3的输出（具体供应商和物料）对大部分安规条款的判定不是必需的。系统允许工序4在工序3完成前启动（这是路线设计时赵总工标记的"并行启动允许"），辅助栏中输入文档区工序3标记为"参考（非必须）"。

8月10日：赵六完成安规认证报告→技术评审通过→文档发布。工序4完成✓。

工序5·实现方案（李四·设计工程师组·8月05日→8月25日）

李四的第二次出场——这次他开发的"实现方案"与之前的"硬件设计方案"是不同的文档类型，由不同的作业指南驱动。两个关键对比：

同一执行者、同一工作中心、不同文档类型：李四在工序2（硬件设计方案）积累的操作经验（TAB菜单的使用、辅助栏的交互模式、检查清单的判定逻辑）在工序5中仍然适用——工艺执行平台的学习曲线在一次任务后大幅下降。但作业指南的结构不同：实现方案的 Step2 检索的是复用构件库（REUSE）而非物料库（MAT）——重心从"选物料"转向"选已有设计模块"。

输入文档链的累积：李四打开工序5工位时，辅助栏输入文档区已有4份已完成文档（用户需求整理+接口定义+硬件设计方案+采购技术规范），合规要求也在7月25日完成。这是整条路线中信息最完整的一道工序——所有前序文档都已就绪。

8月22日，李四在实现方案 Step2 中检索复用构件库（REUSE），发现已有的"电源保护电路模块"可以复用→选定→规则R1（优先复用）自动通过✓。

8月25日：李四完成实现方案→技术评审通过→文档发布。工序5完成✓。

工序6·测试报告（孙七·测试工程师组·8月20日→9月05日）

孙七是第一次出场——测试工程师组与设计工程师组是不同的工作中心。测试报告的作业指南以**编写型步骤**为主（Step2 是"设计测试用例"，Step3 是"记录测试结果"），步骤结构中没有实体库检索。

文档类型的步骤模式差异总结：

工序	文档类型	主导步骤类型	核心孙表使用
工序1	用户需求整理	理解型+编写型	孙表A（输入文档资源）主导
工序2	硬件设计方案	选用型	孙表B（规则清单）主导
工序3	采购技术规范	选用型	孙表B（供应商维度规则）主导
工序4	安规认证报告	合规型	孙表D（合规清单）主导
工序5	实现方案	选用型（复用构件库）	孙表B（复用优先规则）主导
工序6	测试报告	编写型	孙表C（完成标准）主导
工序7	验收报告	编写型+提交型	孙表C（完成标准）主导

孙七在9月03日完成了全部测试用例→[运行全部检查]→全部通过→[提交流转：技术评审]。

9月05日：孙七完成测试报告→技术评审→批准→文档发布。工序6完成✓。

工序7·验收报告（赵六·质量工程师组·9月05日→9月10日）

赵六第二次出场——他在工序4（安规认证报告）积累的经验直接复用。验收报告是路线的最后一道工序：前序6道工序全部完成，6份文档全部在辅助栏输入文档区可见。赵六的工作是汇总所有前序文档的输出、逐项对照验收标准判定是否通过。

最后一道工序的特殊行为：当工序7完成后，系统检测到这是路线的最后一道工序→自动触发业务启动指令的关闭流程→通知钱经理"XYZ电机项目·电源模块 全部工序已完成"→场景业务状态从"进行中"变为"已完成"→路线使用计数+1。

9月10日：赵六完成验收报告→技术评审→批准→文档发布。系统自动：工序7完成✓→业务启动指令 #BI-2026-0089 状态="已完成"→所有工序✓→钱经理收到通知→项目收尾。

第五幕总结：从7月29日到9月10日，5道工序、5位执行者（其中李四和赵六各出场两次）、4个不同工作中心，按路线定义的序列依次推进。每一道工序的执行者打开工位时，辅助栏自动装配好前序文档的已完成版本；每一道工序结束时，系统自动解锁后续工序。整条路线中，执行者不需要知道"这是工序几""前序依赖是谁"——他们只需要打开工位、按作业指南操作、运行检查、完成提交。工艺路线对他完全透明。

十四、回看 —— 执行者、管理者与每个人感知到了什么

以上五幕串联了工艺开发→任务策划→工艺执行→监督评价四个环节的完整协作。以下三个视角分别从执行者体验、管理者监控、和每个角色的完整感知来总结场景的启示。

14.1 场景中各步骤的工位页面对比

理解型步骤（Step 1）的辅助栏——输入文档展开、参考资料可见、无实体库检索、无规则检查菜单：

辅助栏·理解型步骤

当前步骤

Step 1/4

输入文档

用户需求 ✓
接口定义 ✓
合规要求 ⌚

← 重点展开

参考资料

设计规范 [→]
模板 [→]

完成标准

← 折叠

选用型步骤（Step 2）的辅助栏——实体库检索展开、完成标准展开、输入文档折叠：

辅助栏·选用型步骤

当前步骤

Step 2/4

检索→比对→验证

输入文档

← 折叠

实体库检索

[搜索MAT]
当前:§3.2
3个候选
○ LMR14030
○ TPS5430
[比对][选定]
— 或 —
[发起寻源]

← 重点展开

← E2入口

完成标准

← 展开

C1 (2/3)
C2 ⚠
C3 ✓
C4 ?

编写型步骤（Step 3）的辅助栏——参考资料展开、完成标准展开、实体库检索折叠：

辅助栏·编写型步骤

当前步骤	
Step 3/4	
按模板填写	
当前:§1 范围	
参考资料	← 重点展开
设计规范 [→]	
模板 [→]	
数据字典 [→]	
完成标准	← 展开
C1 必填(6/10)	
C2 校验无报错	
C3 规范版✓	
输出流转	
→Step4自检	

提交步骤（Step 4）的辅助栏——完成标准重点突出、输出流转展开：

辅助栏·提交步骤

当前步骤	
Step 4/4	
自检与提交	
完成标准	← 重点突出·全展开
步骤1 ✓ 100%	
步骤2 △ 100%	
步骤3 ✓ 67%	
业务检查清单	
☐ 比价完成	
☒ EOL已评估	
☒ 供应商合	
输出流转	← 展开
提交后触发：	
技术评审	
接收人：王工	
动作：	
通过→批准	
驳回→退回	

14.2 管理者的场景业务视图

钱经理随时打开的场景业务视图——不是工位，是管理看板：

场景业务：XYZ电机项目·电源模块

业务启动指令 #BI-2026-0089

构件：电源模块（自研件）

路线版本：v1.0

创建日期：2026-07-10

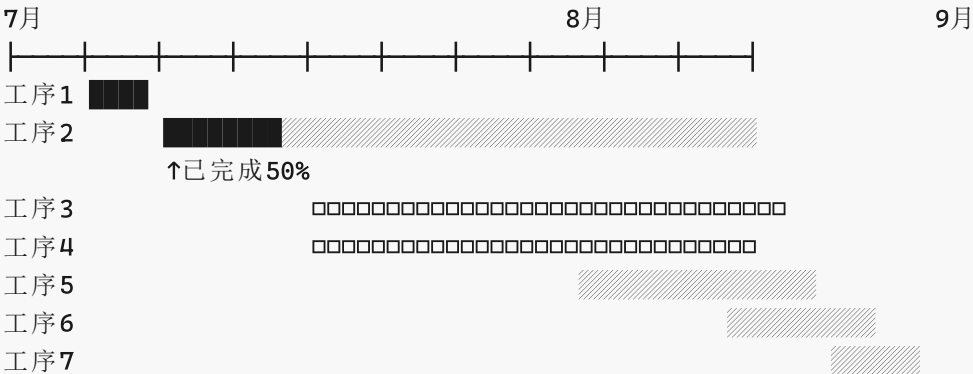
状态：进行中

整体进度：30%

工序状态

工序1	✓ 用户需求整理	张三 · 07-22	进度100%	已发布
工序2	🕒 硬件设计方案	李四 · 08-05	进度50%	编制中 · Step2
⚡ §3.3 EOL预警（已评估） 🕒 §3.2 寻源进行中（#SRC-2026-0089 · 周八）				
检查清单：C1✓ C2⚠ C3✓ C4?				
工序3	□ 采购技术规范	王五 · 08-15	等待前序	
→ 等待：工序2完成				
工序4	□ 安规认证报告	赵六 · 08-10	等待前序	
→ 等待：工序3完成				
工序5	□ 实现方案	李四 · 08-25	等待前序	
工序6	□ 测试报告	孙七 · 09-05	等待前序	
工序7	□ 验收报告	赵六 · 09-10	等待前序	

时间线



阻塞项

⚠ 工序2·§3.2 物料缺失 → 寻源任务 #SRC-2026-0089 处理中
负责人：周八（寻源工程师） 创建：07-23 预计完成：07-30
影响：不阻塞工序2流转（§3.2已标记待就绪），但需在工序2最终发布前完成

[\[查看路线详情\]](#) [\[调整资源分配\]](#) [\[导出进度报告\]](#)

14.3 赵总工、钱经理与执行者们各自感知到了什么

赵总工（工艺工程师）：

- 7月1日：打开构件工艺路线设计器→选定构件→系统推导→调整确认→发布路线v1.0→关闭设计器
- 7月23日：系统通知"李四在工序2触发E2新物料寻源，异常路径E2运行正常"→赵总工看了一眼，不需要处理
- 其余时间：做别的构件的路线设计。整条线两个多月的运转，他不需要参与

钱经理（生产计划员）：

- 7月10日：创建业务启动指令→指定负责人→启动
- 随时打开场景业务视图：7道工序的进度条、每份文档的进度值（自动计算）、阻塞项标记
- 7月23日：看到工序2进度从30%突然跳到50%（李四触发了寻源），C4待确认→联系李四确认选型→C4通过
- 不需要追问任何人"做到哪了"——进度是系统自动算的

张三（工序1·执行者）：

- 7月11日收到通知→打开→做完→7月22日完成
- 感知到的：任务出现→工位→完成→任务消失
- 不知道的：自己的文档是"工序1"，后面还有6道工序，赵总工编过路线

李四（工序2·执行者）：

- 7月23日收到通知→打开→辅助栏已有张三的文档→操作→触发寻源→完成→提交
- 7月30日收到通知→回到文档→更新\$3.2→完成
- 感知到的：任务出现→工位已装配好输入文档→做→完成→（7月30日）回流更新
- 赵总工编制的路线、钱经理锁定的版本——对他完全透明

王五（工序3·执行者）：

- 7月29日收到通知→打开→辅助栏已有张三和李四的文档
- 7月30日收到通知→输入文档已更新→打开查看→最新的硬件设计方案v1.1已在辅助栏中
- 不需要去问"李四你的文档写好了没" "李四你的文档更新了没"——系统自动装配最新版本

第六部分：滚雪球法 —— 组织如何构建、运行与演化

EOS

前五部分回答了 EOS 是什么、为什么、怎么设计。这一部分给出落地的方法论——**滚雪球法**。

核心逻辑：不是先建好基础设施再用 EOS，而是在用 EOS 的过程中让方法和数据像滚雪球一样自然生长。从一片雪花开始——一份个人检查清单、一条只有3道工序的路线、第一个项目留下的几条数据。每执行一次，雪球滚一圈，粘上更多东西——路线被验证了一次、作业指南被迭代了一次、实体库多了几条被验证的物料、Owner多了一次检视记录。越滚越大，越滚越不需要外力。

三个滚动阶段：**启动期**（从-1到0.1）——当没人、没时间、没数据时，雪球怎么从零开始滚。**扩展期**（从0.1到1）——试点跑通后，雪球怎么滚到同类构件、滚出Owner制度、滚过第一次信任危机。**自治期**（从1到N）——演化态启动后，雪球怎么自己滚下去，不再依赖最初的推动者。每节一个动作：触发条件→谁来做→做什么→交付物→验收标准。

十五、启动阶段 —— 从 -1 到 0.1

§2.5 矩阵定位：启动阶段聚焦**产品开发层**——让最下面一行的四个环节（工艺开发→任务策划→工艺执行→监督评价）在一个构件上首次跑通。不求全、不求好、不求系统——只求验证“工艺化是否比不工艺化更好”。

滚雪球的起点永远是一个构件。不要试图同时为多个构件建路线——从一个开始，让它滚起来。

15.1 启动前自我诊断

在投入任何精力之前，先用5个问题判断组织的起点位置。如实回答——不要“应该是”，要“实际是”。

问题	判断	如果答案为“否”
组织中是否存在一个重复性知识工作活动，目前完全依赖某个人？	是/ 否	没有痛点就没有动力。先观察，不启动
那个人是否愿意花15分钟讲讲“自己怎么做”？（不需要写，只需要讲）	是/ 否	走15.2零条件路径A：自己给自己写清单
是否有至少一个2-5年经验的人，能独立完成该活动但质量不稳定？	是/ 否	试点执行者不存在。走15.2路径B：从复盘反推

问题	判断	如果答案为"否"
该活动的输出（文档、选型结果等）是否能被客观检查？（有明确的"对/错"标准）	是/否	无法工艺化。换一个活动，或先从能客观检查的部分做起
未来3个月内是否有至少一个该类型的项目要启动？	是/否	没有跑第一个项目的机会。先准备路线，等窗口

诊断结果： 5问全部为"是" → 走15.3正式试点。有"否" → 走15.2零条件启动对应路径。3个以上"否" → 组织尚未具备启动的最低条件，先观察和等待窗口，不要强推。

15.2 零条件启动：三条不需要任何人批准的路径

当诊断不通过时，不等待、不抱怨、不写方案要资源。选一条路径，一个人就能开始。

路径A：给自己写清单（触发：有人愿意讲，但没人有能力做执行者）

步骤	动作	耗时
1	下一个项目开始前，回顾上次同类项目踩的坑，写3-5条"这次先确认"的检查项	15分钟
2	项目执行中，逐条对照	嵌入正常工作流
3	项目完成后，保留有用的、删掉没用的、补充新发现的	10分钟
4	重复3个项目周期	—

交付物： 一份经过3个项目验证的个人检查清单。**验收标准：** 第3个项目执行者主观感觉比第1个更省力。

路径B：从复盘反推（触发：没人有能力做执行者，且连15分钟写清单的人都没有）

步骤	动作	耗时
1	找一个刚交付的同类项目，问执行者："哪个环节最折腾？"	15分钟
2	把"折腾点"倒推为"下次怎么避开"——这就是第一版检查项	15分钟
3	下一个同类项目开始前，交给执行者试用	嵌入正常流程
4	项目完成后反馈：有用保留，无用删除	10分钟

交付物： 一份从真实痛点反推的检查清单。**验收标准：** 执行者确认"下次还用"。

路径C：一个人自导自演（触发：所有诊断都是"否"，但你相信这件事是对的）

如果你自己就是那个"做得最好的人"——给自己写清单（路径A）。如果你有一份检查清单，下一个项目自己用了觉得省力——分享给一个同事试试。如果同事也觉得省力——你们已经有了一个最小可行工艺的雏形。不需要"试点"这个标签——野蛮生长比正规试点更顽强。

AI赋能： 三条路径中，AI可以显著降低启动门槛。路径A——把上一次的项目复盘录音转成文字，让AI从中提取"踩过的坑"并生成第一版检查清单草稿，人只需审核修改。路径B——把复盘访谈的要点输入AI，让AI反推"下次怎么避开"的检查项。路径C——AI帮助个人生成清单后，还能将其自动格式化为团队可共享的版本。**AI把"写"的成本降到接近于零，人只需要做"判断"——这恰好是'工'（AI可做的）和'艺'（人必须做的）在启动阶段的分工。**

15.3 第一个正式试点的执行手册

触发条件： 诊断5问全部为"是"，或零条件路径中出现了"同事来问你怎么做的"。

角色分配：

角色	谁来做	投入时间
推进者	对EOS落地有动力的人（不一定是领导）	全程跟进，约10%带宽
工艺开发者	做得最好的那个人，或从路径A/B中自然浮现的人	第1-2周约4小时，之后递减
执行者	2-5年经验、能独立完成但质量不稳定的人	正常项目投入，不超过正常工时
策划者	该项目的项目管理者	正常策划投入+30分钟绑定路线

周级动作表：

周次	动作	交付物	验收
第1周	推进者+工艺开发者访谈最优实践者（1小时）。工艺开发者整理出路线骨架（2-3道工序）+每道工序的步骤序列和关键检查点	路线v0.1 + 作业指南v0.1（只有步骤+检查点+出入口）	最优实践者看过初稿，确认"这是我的做法"
第2周	工艺开发者在路线设计器中录入路线（如果平台未就绪，用Excel/Markdown代替）。推进者确认策划者、执行者就位	路线已录入（或临时文件已就绪）	策划者确认"我知道怎么绑定这条路"
第3-6周	第一个项目在路线上运行。工艺开发者第一周每天花10分钟看执行者反馈，之后每周一次	执行过程记录（人工记录，如果系统未就绪）	执行者没有提出"我不想按这个做"
第7-8周	复盘。推进者主持，执行者+工艺开发者参加	复盘报告（1页纸）	见15.4验收标准

如果平台未就绪： 路线用Excel管理（列：工序序号/文档类型/检查清单/前序依赖），作业指南用Markdown写，检查结果人工记录。第一个项目不需要任何系统——需要的是验证"工艺化是否比不工艺化更好"。

AI赋能： 工艺开发者访谈最优实践者后，将访谈录音或笔记输入AI，让AI生成路线骨架（工序序列+每道工序的步骤描述+建议的检查点），人工审核调整后即可发布v0.1。第7-8周复盘时，将执行

者的反馈记录输入AI，AI自动提炼为"改进建议列表"，工艺开发者逐条判断采纳/不采纳。**AI将工艺开发者的时间从'写'压缩为'审'——第一版路线从几天缩短到几小时。**

15.4 试点验收：4项通过标准 + Go / No-Go / Adjust 决策

序号	验收项	判定方法	通过线
①	执行者省力	执行者主观评价："比自己摸索更省力 / 差不多 / 更费力"	"更省力"或"差不多"
②	质量不降	检查项通过率 vs 执行者之前3个同类项目的检查通过率（人工估算即可）	≥ 之前平均水平
③	资产产出	项目完成后，是否留下了可被下一个同类项目复用的东西（路线文件、检查清单、选型记录）	≥ 1项可复用资产
④	执行者未绕路	执行过程中，执行者是否有绕过工艺的行为（跳过检查、不记录异常）	绕过次数 = 0

决策矩阵：

①省力	②质量	决策	下一步
✓	✓	Go	启动第二个同类构件，走\$十六扩展
✓	X	Adjust	不扩展。修作业指南——检查点可能设错了。修完再跑一个项目验证
X	✓	Adjust	不扩展。修作业指南——步骤可能太繁琐。简化后再跑
X	X	No-Go	停止。根因可能是：工艺开发者误解了最优实践、执行者不配合、构件选错了。回到诊断（\$15.1）重新判断

十六、扩展阶段 —— 从 0.1 到 1

\$2.5 矩阵定位：扩展阶段在**产品开发层**已验证的基础上，开始触达**工艺开发层**的第一格——路线模板的衍生和变异（工艺开发层的"工艺开发"列）。Owner 制度的启动（\$16.2）为工艺开发层的"监督评价"列（路线运行效果追踪）埋下第一个锚点。

16.1 扩展决策：什么时候从1个构件到1类构件

触发条件： 试点验收Go，且以下三个条件全部满足。

条件	判定
同一类别中，存在至少2个构件可以用相同或相似的路线模板	查看构件分类表，同PL3类型下是否有其他构件
试点路线的执行人反馈稳定为"省力"（连续2个项目，不是偶然1次）	问执行人："如果下一个同类项目还让你按这个做，你愿意吗？"
工艺开发者有时间维护第二、第三条路线的衍生版本	工艺开发者确认：当前负载允许增加1-2条路线的Owner职责

扩展动作表：

步骤	动作	负责
1	工艺开发者在试点路线基础上，用变异式方式为新构件生成候选路线（调整差异部分）	工艺开发者
2	策划者为新构件的第一个项目绑定路线，指定执行人	策划者
3	两个项目结束后，评估是否可以将该构件类别的路线抽取为"路线模板"	工艺开发者+推进者

何时停止扩展： 工艺开发者的Owner职责超过3条路线时，必须先培养第二个工艺开发者。一个人的Owner上限是3条——超过这个数，检视质量必然下降。

16.2 Owner制度启动包

触发条件： 第一条路线发布后立即启动。

任命模板（1页，推进者+Owner共同签署）：

路线名称：【构件类别】-【版本号】

Owner：【姓名】

备选Owner：【姓名】

首次数据检视日期：【不晚于第一个项目完成后2周】

季度检视频率：每季度1次，每次不超过2小时

Owner承诺：

- 我每季度至少检视一次路线运行数据
- 当数据出现异常信号时，我48小时内做出初步判断
- 我每年至少与执行这条路线的执行人对话2次
- 备选Owner参与我的每一次检视

推进者承诺：

- 确保路线Owner在季度绩效评估中有时段被分配用于路线检视（不低于每季度2小时）
- 当Owner发出异常信号时，24小时内协调相关方响应

签署：【Owner】【推进者】【日期】

上岗检查清单（Owner完成首条路线发布时逐条确认）：

- ☐ 我理解这条路线的设计逻辑（为什么是这些工序、这个顺序、这些检查点）
- ☐ 我已向备选Owner当面说明职责，对方表示理解和接受
- ☐ 我知道路线运行数据从哪里看（系统仪表盘 or 人工统计表）
- ☐ 我已在日历上标记了首次数据检视的日期
- ☐ 我已与第一个项目的执行者沟通过异常路径的使用方式

AI赋能： Owner首次数据检视时，将原始运行数据输入AI，让AI自动生成"检视摘要"——哪些指标偏离了基线、哪些异常路径触发频率在上升、哪些检查项的不通过率有系统性模式。Owner不再需要从原始数据中手动翻找信号，AI替他完成"信息化→数字化"的跳跃。**AI不是替代Owner的判断——是把Owner的时间从'找信号'释放为'判归因'。**

16.3 第一次信任危机的处理流程

触发条件： 执行者按作业指南操作后，结果不如他自己独立完成的水平——或者执行者口头表达"这个工艺不靠谱"。

处理流程（24小时内启动）：

步骤	动作	负责	时限
1	执行者记录：具体哪个步骤/检查点导致了问题？实际结果和预期差了多少？	执行者	当天
2	工艺开发者+执行者当面过一遍：是工艺写错了，还是执行理解有偏差，还是特殊情况？	工艺开发者	24小时内
3	判断： ☐ 工艺问题 → 修订作业指南，当前项目走异常路径继续 ☐ 理解偏差 → 补充作业指南的说明 ☐ 特殊情况 → 记录在案，不修订，但标记为"下次同类情况参考此例"	工艺开发者	24小时内
4	推进者确认：修订后执行者是否接受？是否愿意继续按修订后的工艺走？	推进者	48小时内

红线： 如果执行者按工艺做导致的结果比他自己独立做差——承认工艺有问题，当场修订。不要用"工艺是经过验证的，你要相信工艺"去压执行者——这会永久破坏信任。**信任破产的项目，EOS在这个组织里活不过半年。**

16.4 惯性突破的四个锚点

触发条件： 试点验收Go后30天内。

锚点	动作	负责	时限
锚点1：可见性	在管理者视图（或周报/项目看板）中，试点构件标注"工艺化"，相邻无路线构件形成对比	推进者	试点验收后1周内

锚点	动作	负责	时限
锚点2：绑定	策划者在下一个同类构件的业务启动指令中绑定试点路线——不是"建议"，是默认选项	策划者	扩展启动时
锚点3：Owner	Owner完成首次数据检视，输出检视记录（哪怕半页纸）	Owner	第一个项目完成后2周内
锚点4：复用	第二个同类构件用变异式方式从试点路线衍生——不是重新设计	工艺开发者	扩展启动时

30天检验： 如果30天内这四个锚点全部落地——EOS在这个构件类别上开始扎根。如果任何一个锚点延迟超过30天——回到§15.1诊断，重新判断是否具备扩展条件。

十七、自治阶段 —— 从 1 到 N

§2.5 矩阵定位： 自治阶段正式进入**工艺开发层**的全排建设——演化态闭环启动（工艺开发层的"监督评价"列：路线运行效果的数据回流和检视），并开始触达**方向选择层**（"这条路线的设计策略对不对"的判断从直觉升级为数据驱动）。五个阶梯（§17.5）对应 §2.5 矩阵从产品开发层→工艺开发层→方向选择层的逐级爬升路径。

17.1 演化态启动的判断标准

演化态是§一.6定义的核心机制——运行态产生过程数据，演化态驱动工艺改进。但它不会自动启动。需要有人决定"从什么时候开始，我们不再只是看进度，而是看数据"。

触发条件（三个全部满足）：

条件	判定方法
至少有1条路线被3个以上项目使用过	路线使用次数 > 3
至少有1个Owner完成了首次数据检视并输出了检视记录	检视记录存在
推进者或主任工艺师已就位	有人对演化态闭环运转负有责任

启动动作表：

步骤	动作	负责	交付物
1	为每条活跃路线建立数据基线：当前的平均周期偏差、检查项通过率、异常触发频率	Owner	路线基线报告（半页纸/路线）
2	设定信号阈值：什么情况算"需要关注的异常"	Owner+主任工艺师	阈值定义（如"连续3个项目同一检查项不通过→信号"）

步骤	动作	负责	交付物
3	建立季度检视日历：每条路线的Owner每季度固定一个2小时时段用于数据检视	主任工艺师	年度检视日历
4	EOS运维者确认数据通道畅通：Owner能看到数据、异常信号能到达Owner	EOS运维者	数据通道确认

验收标准： 一个季度后，至少80%的活跃路线Owner完成了检视并输出了检视记录。

17.2 路线季度检视与修订流程

触发条件： 演化态已启动，Owner按检视日历执行。

检视模板（每条路线每季度填一次）：

项目	内容	数据来源
本季度使用次数	[填数字]	系统/人工统计
平均周期偏差	工序1：[] 工序2：[] 工序3：[]	系统/人工统计
检查项不通过率Top3	1. [] 2. [] 3. []	孙表C数据
异常触发统计	E1：[]次 E2：[]次 E3：[]次 E4：[]次	孙表E数据
执行者反馈	本季度收到[]条反馈，采纳[]条	工艺质疑记录
是否需要修订	☐ 是（→启动修订流程） ☐ 否	Owner判断
备选Owner是否参与	☐ 是 ☐ 否（下周安排）	Owner确认

修订决策树：

数据信号出现
- 是工艺问题？（如：检查项标准过严导致正常产出被判不通过） → Owner修订作业指南 → 走版本管理（§五.3-5.4） → 下次项目生效 - 是执行问题？（如：某个执行者连续跳过检查清单，但其他人正常） → 推进者/监督者与该执行者沟通 → 不修订工艺 - 是策划问题？（如：周期偏差因策划时给的时间太紧，与工艺无关） → 反馈策划者 → 不修订工艺 - 归属不明？（数据信号存在但无法判断归因） → Owner标注"观察中" → 下季度再判 → 连续2个季度归属不明 → 升级到主任工艺师

修订后追踪： 每次修订后，下一个使用该路线的项目作为"修订验证项目"。该项目结束时，Owner检查修订效果——指标是否改善。记录在修订历史中。

AI赋能： 季度检视模板中的数据（使用次数、周期偏差、检查项不通过率、异常触发频率）直接输入AI，AI自动生成"信号摘要"——标注超出阈值的指标、识别跨季度的恶化趋势、对比同类路线的数据表现。修订决策树中"归属不明"的分支——AI可以给出初步归因建议（基于历史数据的相关性分析），Owner在此基础上做最终判断。**AI从不'替代决策'——它把Owner从'找问题'升级为'判问题'。**

17.3 数据健康度季度自检

触发条件： 演化态已启动，EOS运维者每季度执行。

自检清单：

检查项	方法	警戒线	当前值	行动
实体库"休眠"数据占比	统计>1年未被引用的物料/构件/供应商条目占总条目比例	>30%	[]	通知域工艺师，标记休眠数据，下次使用前需验证
作业指南版本一致性	逐条路线检查：路线中引用的作业指南版本是否=当前有效版本	不一致条目 > 0	[]	通知Owner，评估是否需要升级路线版本
过程数据回流通道	抽查最近完成的1个项目：其执行数据是否已出现在Owner可查看的范围内	数据缺失	[]	EOS运维者排查通道，24小时内修复
Owner检视完成率	本季度应检视的路线中，已完成的占比	<80%	[]	主任工艺师跟进未完成Owner，了解障碍

交付物： 季度数据健康度报告（1页），抄送主任工艺师+推进者。

AI赋能： 数据健康度自检的四项检查中，休眠数据占比和版本一致性可由AI直接连接数据源自动计算并标注异常项；过程数据回流通道可通过AI定时抽查自动验证。AI每季度自动生成自检报告初稿，EOS运维者审核补充后即可发出。**季度自检从半天人工对数据缩短到30分钟审核AI报告。**

17.4 组织治理结构的落地包

触发条件： 演化态已启动，且活跃路线≥3条，或Owner≥3人。

角色任命模板：

角色	从谁中产生	任命方式	投入时间
主任工艺师（1人）	第一批Owner中经验最丰富、数据检视最持续的人	推进者提名+Owner互认	每季度2-4小时
域工艺师/路线Owner（每域1~N人）	本域构件的最优实践者，或从备选Owner晋升	主任工艺师提名+推进者确认	每季度每路线2小时

角色	从谁中产生	任命方式	投入时间
EOS运维者（1人）	对系统和数据敏感的人，不一定是技术背景	推进者指定	每季度2-4小时

三个角色的季度例行动作：

角色	Q1	Q2	Q3	Q4
主任工艺师	审核路线模板，检查跨构件一致性	跟进未完成检视的Owner	协调Owner间经验交流	主持年度"从头设计"审查 + 下年度检视日历
路线Owner	季度数据检视+检视记录	季度数据检视+检视记录	季度数据检视+检视记录	季度数据检视 + 年度"从头设计"审查
EOS运维者	数据健康度自检	数据健康度自检	数据健康度自检	数据健康度自检 + 年度数据治理建议

会议日历模板（每年底排定下一年）：

- 季度检视窗口：每年1月/4月/7月/10月的第一周
- 年度"从头设计"审查：每年12月
- Owner经验交流会：每年6月（非强制，有议题才开）

KPI模板（写入对应角色的季度绩效目标）：

- 路线Owner：所负责路线的季度检视完成（是/否）+ 执行质量偏差变化趋势（缩小/持平/扩大）
- 主任工艺师：活跃路线Owner检视完成率（≥80%）+ 跨构件路线一致性抽查（年度）
- EOS运维者：数据健康度季度自检完成（是/否）+ 数据通道可用性（年度可用率≥95%）

17.5 五个阶梯的总览

EOS在组织中的生长不是一步到位，而是沿五个阶梯逐级爬升。每一级以前一级为基础。

制度化 → 流程化 → 信息化 → 数字化 → 智能化

阶梯	核心动作	当前阶段关联
制度化	路线绑定策划、Owner任命、数据不可篡改	启动-扩展期（\$十五~十六）
流程化	四个环节运转、路线模板复用、作业指南迭代	扩展期（\$十六）
信息化	资源自动装配、检查结果自动汇总、进度自动可见	平台就绪后自然升级
数字化	数据结构化采集、跨项目关联分析、异常模式自动识别	自治期（\$十七）启动后的长期方向
智能化	系统主动提醒、AI辅助工艺改进	数字化完成后的远景目标

AI在五个阶梯中的角色演进：

阶梯	AI的角色	典型场景
制度化	辅助者	帮助起草路线和作业指南初稿，人做审核和决策
流程化	协作者	在执行者操作时实时提示检查点、自动校验规则
信息化	记录者	自动采集和汇总过程数据，消除人工填报
数字化	分析者	跨项目关联分析、异常模式识别、归因建议
智能化	建议者	基于历史数据主动推荐路线改进、预测项目风险

永远不要跳级。 没有制度化的流程化是空转，没有信息化的数字化是手工报表，没有数字化的智能化是随机猜测。AI也一样——没有高质量过程数据之前，AI的"智能建议"只是随机猜测的加速版。

十八、总结

底层概念

工艺（**Process**）—— 方法的体系化，从个体手艺到组织资产的转化

- ├ 工 —— 可外化、可传递、可检验、可重复的方法描述（资产）
- └ 艺 —— 还做不到以上四点的方法能力（矿）
 - └ 工从艺炼：艺→转化→工，环境变化→退化→新艺→新转化

一个 **process**，三个侧面：

- 过程（实际发生的轨迹·不可修改·不可复用）—— 流转卡
- 流程（设计的步骤序列·可修改走版本·可复用）—— 工艺路线
- 工艺（每一步的方法定义·可修改走版本·可复用）—— 工序卡片/作业指南

组织化运行：工艺开发 → 任务策划 → 工艺执行 → 监督评价（四个环节）

- 间隙①（通用方法如何适配具体项目）→ 策划者的艺来消化
- 间隙②（计划在现实中总会偏离）→ 执行者的艺来消化
- 间隙③（偏差归因有歧义）→ 监督者的判断力来消化

两种形态：运行态（单次任务实例）⇌ 演化态（跨任务工艺改进）

核心概念对应表

制造业	EOS
产品（一个物料号）	构件（一个设计态构件）
工艺路线（ Routing ）	构件文档开发路线（主业务数据）
工序卡片（ Operation Sheet ）	作业指南（子业务数据·做→查→过）
工步（ Sub-operation ）	作业指南中的一个步骤
工作中心（ Work Center ）	能完成该文档类型开发的人员集合
工位（ Workstation ）	执行者的个人工作台

流转卡 (Traveler)	任务实例 (路线×业务启动指令)
CAPP	构件工艺路线设计器
工艺工程师	构件设计负责人
生产订单	业务启动指令

三层业务数据总结

主：构件文档清单（构件独有）——定义需要产出哪些文档、顺序、工作中心
子：文档作业步骤清单（可跨构件复用）——定义每份文档怎么做（做→查→过）

孙A：步骤资源清单 —— 需要什么（装配用）
孙B：步骤规则清单 —— 做得对不对（校验用）
孙C：步骤检查清单 —— 做完了没有（步骤出口闸门）
孙D：步骤合规清单 —— 合不合外部规矩（人判断）
孙E：步骤异常清单 —— 出问题怎么办（决策用）

执行者工位布局

顶部: 任务上下文 (场景业务进度+本文档步骤条+截止日期)

正文 (封面表单+TAB·含审核下拉菜单·+正文编辑区)

辅助栏

- 资源清单
- 完成标准
- 输出流转
- 协作

关键原则

范式层（第一部分）：

1. **工艺的本质是组织资产。** 把方法从人身上剥离，固化为可传承、可审查、可优化的知识对象。
2. **工从艺炼，艺不消失。** 工艺开发是专门的"艺→工"转化活动。艺是工的前沿矿藏，消灭艺就是消灭未来的工。
3. **过程≠流程。** 流程是设计出来的（可复用模板），过程是发生出来的（不可复用记录）。锁定的是流程，回流的是过程。
4. **四个环节缺一不可。** 工艺开发（面向未来）→任务策划（未来到当下的转换）→工艺执行（面向当下）→监督评价（过去与未来之间）。任务策划是最容易被忽略但至关重要的一环——通用方法需要通过它才能实例化为具体任务。
5. **运行态与演化态是两个闭环。** 运行态闭环已设计（做→查→过→异常回流），演化态闭环待建设（过程数据回流→工艺修订）。
6. **生产过程是骨架，不是全部。** 生产过程负责可靠性，其他范式（委托指挥/同行生产）在各自适用区补充适应性、创造性。

设计层（第二~五部分）：

1. **工艺路线的组织轴心是工序（一份文档），不是工步。** 构件的工艺路线是一组文档的序列。作业指南是其中一份文档的开发指南，里面的步骤是工步。
2. **一个工序对应一个工作中心，工作中心下有多个工位。** 工序声明"由哪类角色完成"，不指定具体人。
3. **路线版本锁定在业务启动指令创建时。** 已创建的任务不受后续路线升级影响。
4. **TAB步骤审核菜单是作业指南孙表的UI投影。** 结构随步骤类型自动切换，不需要设计者手工配置。
5. **资源清单上的每一项按资源类别提供下钻行为。** 辅助栏是所有资源的统一查看器。
6. **五张孙表结构不同、平台处理方式不同、工位上的表现也不同。** 它们不该混在一张表里。
7. **执行者对路线的感知不是一张表，而是一条任务流。** 工艺路线对他完全透明。